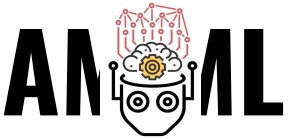
Politecnico di Torino

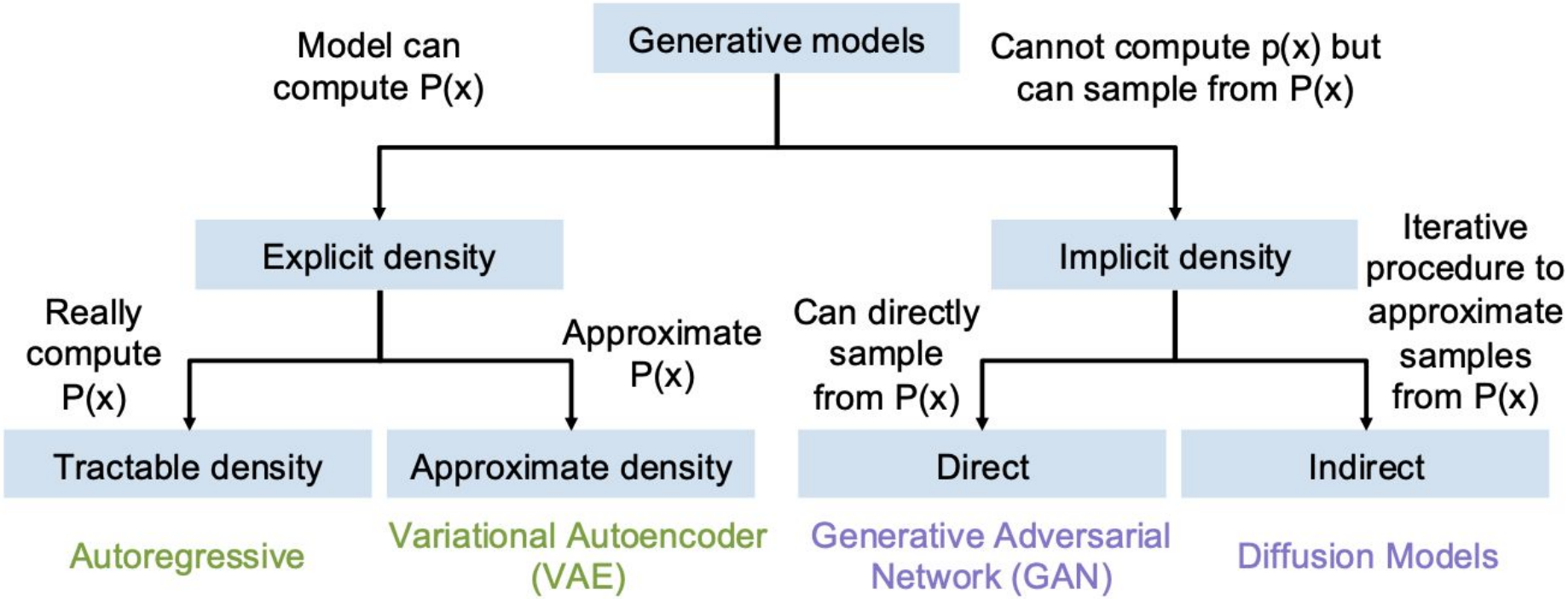


Advanced Machine Learning

A.A. 2025/2026

Tatiana Tommasi

Taxonomy of Generative Models



Slide credit: Justin Johnson

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \ldots x^{(N)}$, train the model by solving:

$$W^* = \arg\max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

$$= \arg\max_W \sum_i \log p(x^{(i)})$$

Log trick to exchange product for sum

$$= \arg\max_W \sum_i \log f(x^{(i)}, W)$$

This will be our loss function!
Train with gradient descent

Slide credit: Justin Johnson

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x consists of multiple subparts:

$$x = (x_1, x_2, x_3, \ldots, x_T)$$

Break down probability using the chain rule:

$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \ldots, x_T) \\ &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \ldots \\ &= \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}) \end{aligned}$$

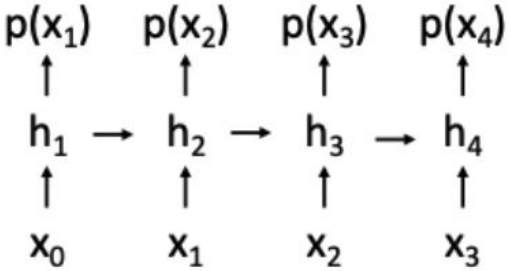
Probability of the next subpart given all the previous subparts

Slide credit: Justin Johnson

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

We have already seen this!



Language modeling with RNN

$$x = (x_1, x_2, x_3, \dots, x_T)$$

$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots \\ &= \prod_{t=1}^{T} p(x_t \mid x_1, \dots, x_{t-1}) \end{aligned}$$

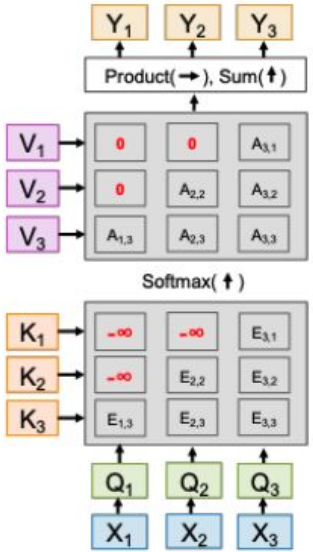
Probability of the next subpart given all the previous subparts

Slide credit: Justin Johnson

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Language modeling with masked Transformer



$$x = (x_1, x_2, x_3, \dots, x_T)$$

$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots \\ &= \prod_{t=1}^{T} p(x_t \mid x_1, \dots, x_{t-1}) \end{aligned}$$

Probability of the next subpart given all the previous subparts

Autoregressive Models of Images

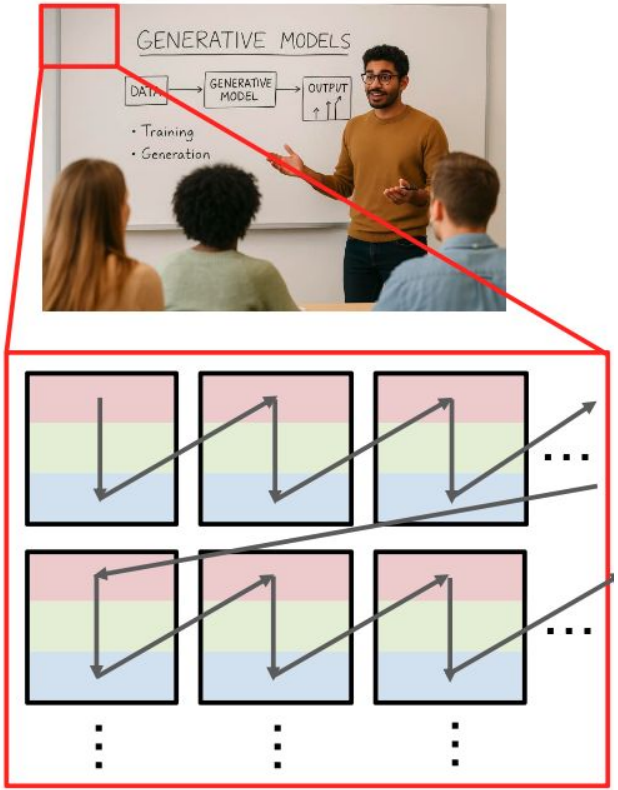
Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values [0…255]

Model with an RNN or Transformer

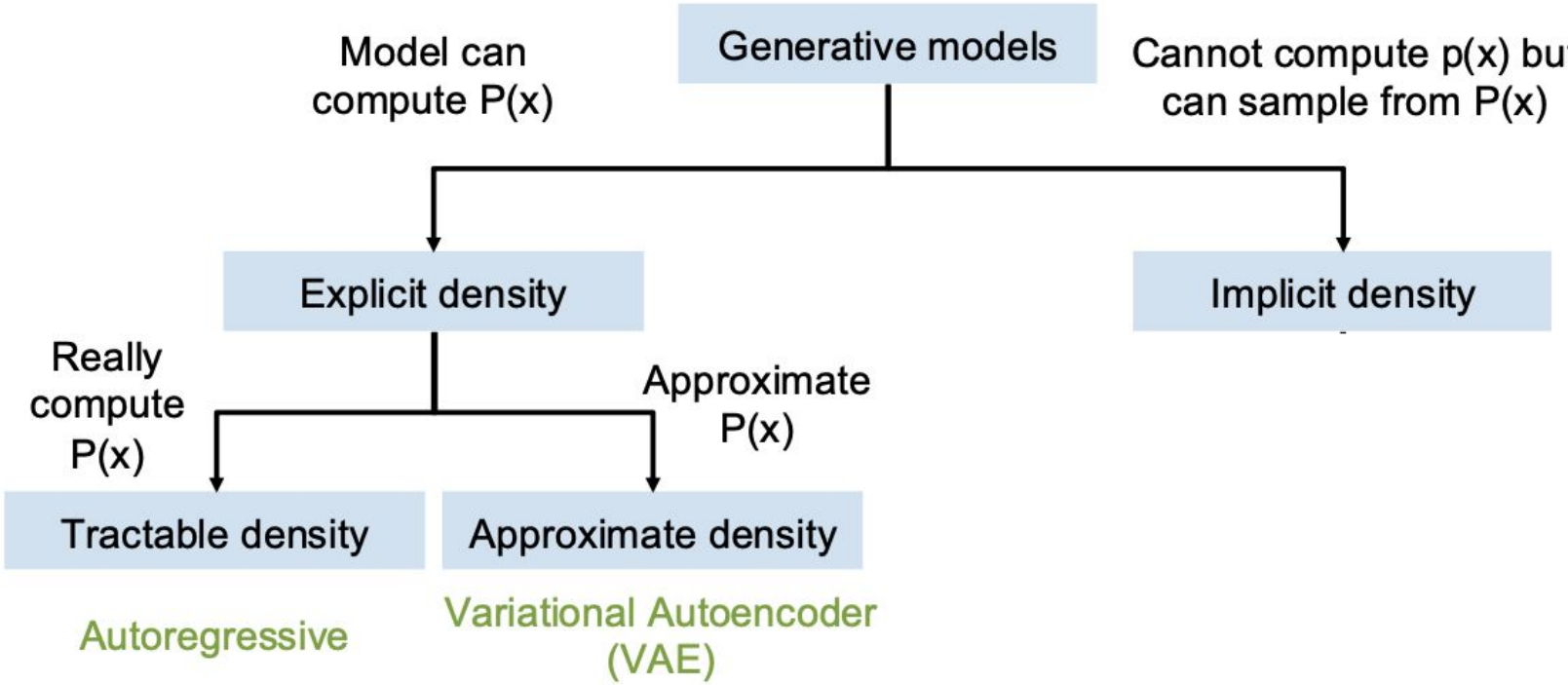
Problem: Too expensive. 1024x1024 image is a sequence of 3M subpixels

Solution (jumping ahead): Model as sequence of tiles, not sequence of subpixels



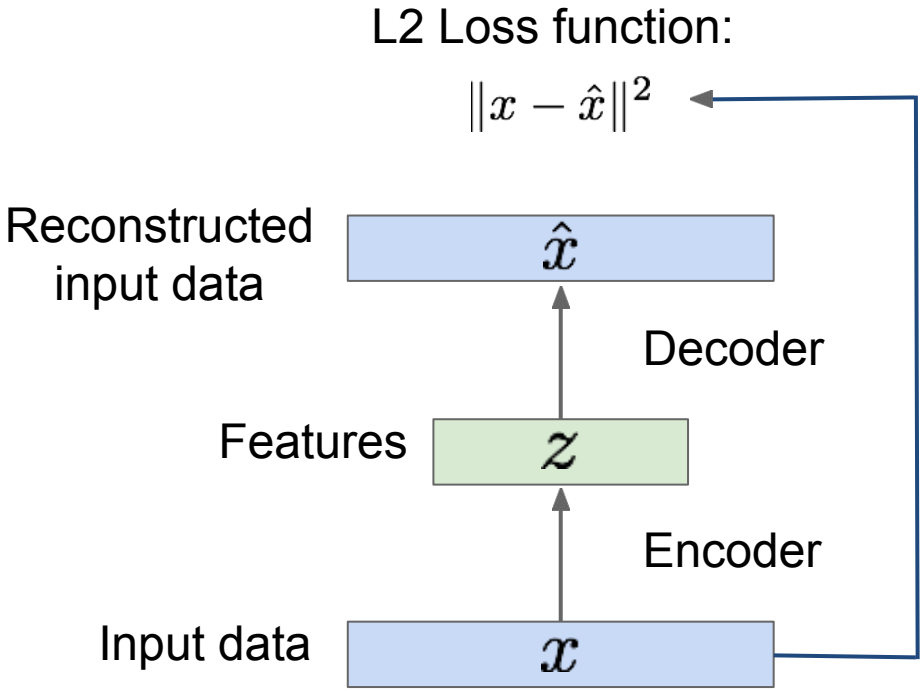
Slide credit: Justin Johnson

Taxonomy of Generative Models



Slide credit: Justin Johnson

Some background first: AutoEncoders

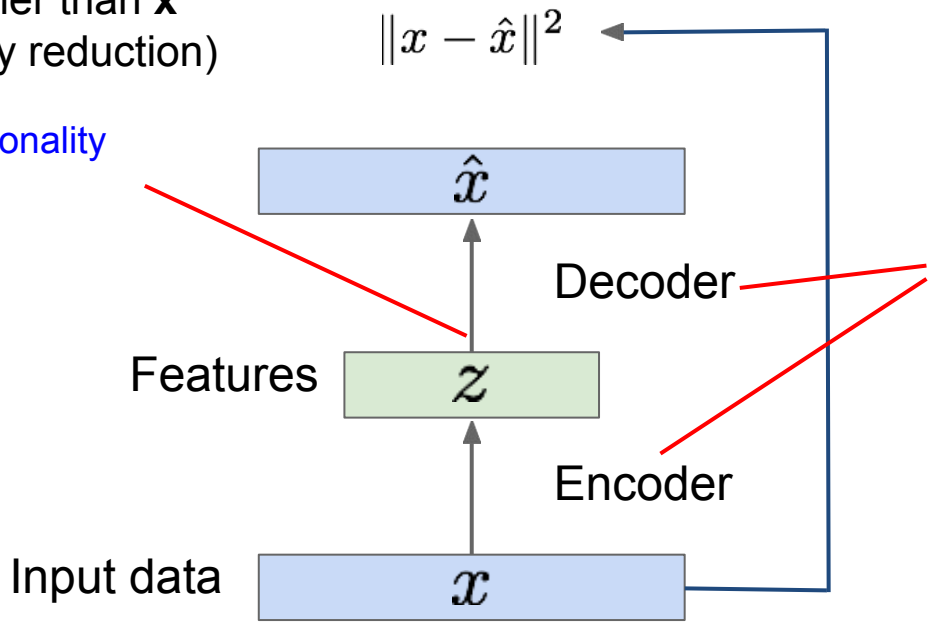


Some background first: AutoEncoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

z usually smaller than **x** (dimensionality reduction)

Q: Why dimensionality reduction?



Originally: Linear + nonlinearity (sigmoid)
Later: Deep, fully-connected
Later: ReLU CNN (down / up conv)
Later: Transformers

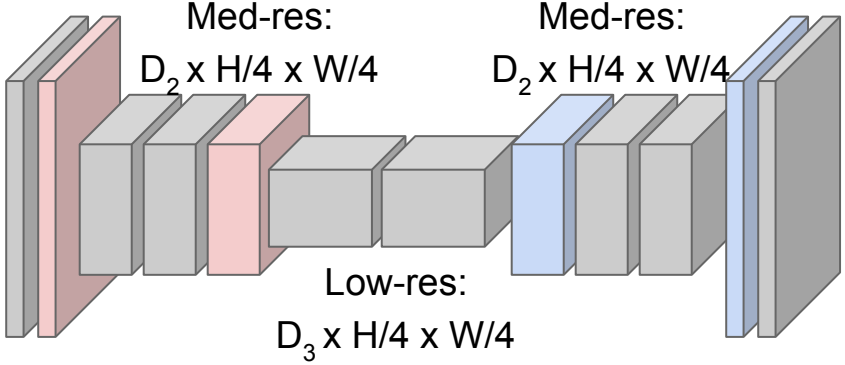
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling:
???

Input:
3 x H x W



High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2

Predictions:
H x W

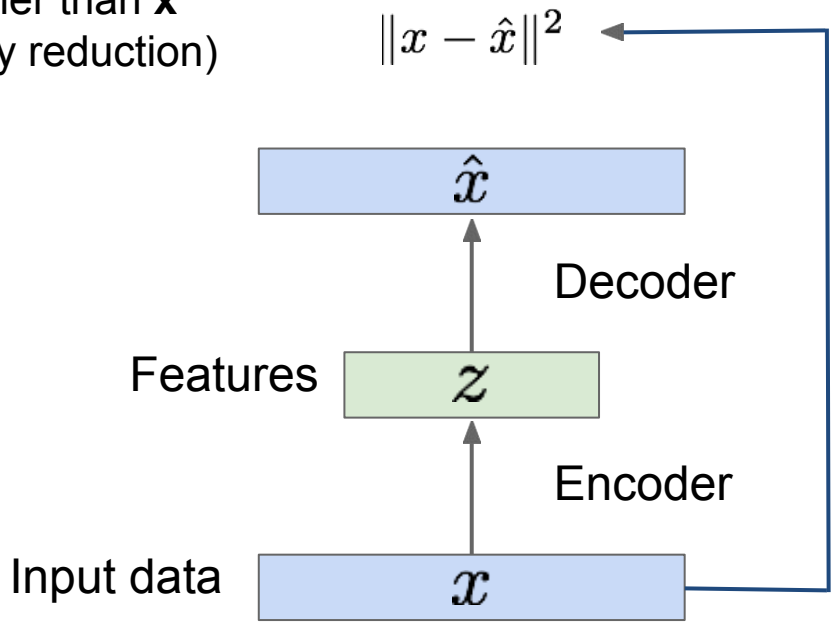
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Slide credit: Fei Fei Li

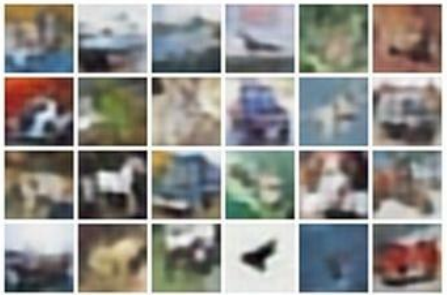
Some background first: AutoEncoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

z usually smaller than **x**
(dimensionality reduction)



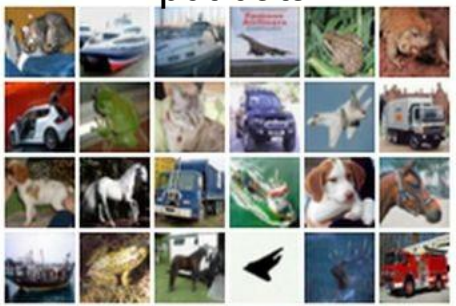
Reconstructed data



Decoder: 4-layer upconv

Encoder: 4-layer conv

Input data

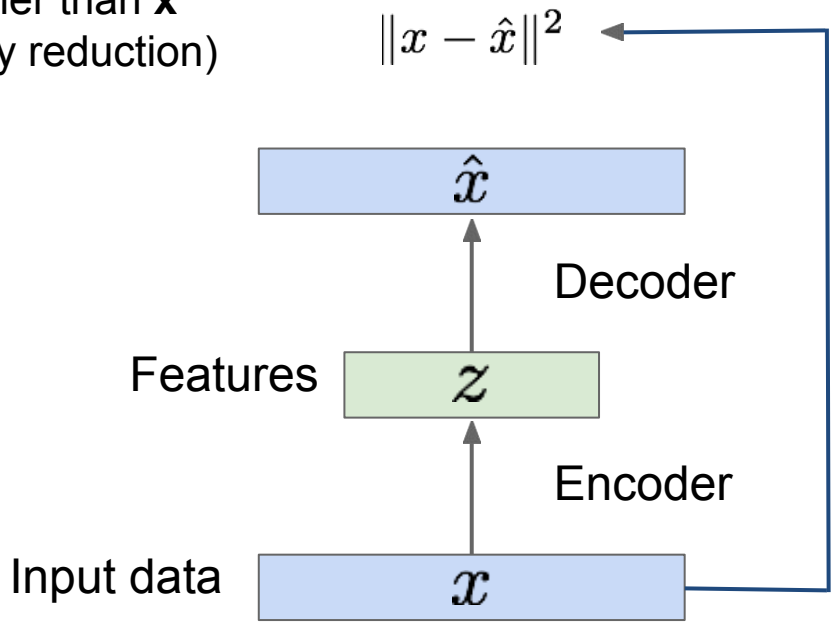


Some background first: AutoEncoders

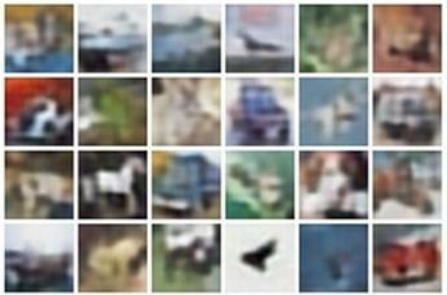
Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

Doesn't use labels!

z usually smaller than **x**
(dimensionality reduction)



Reconstructed data

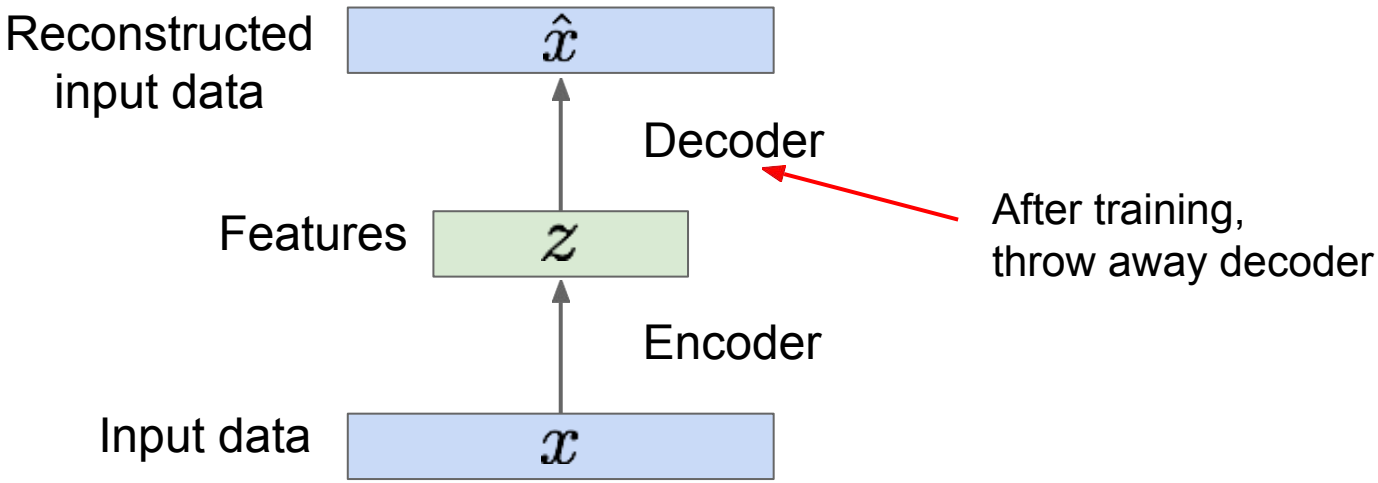


Decoder: 4-layer upconv

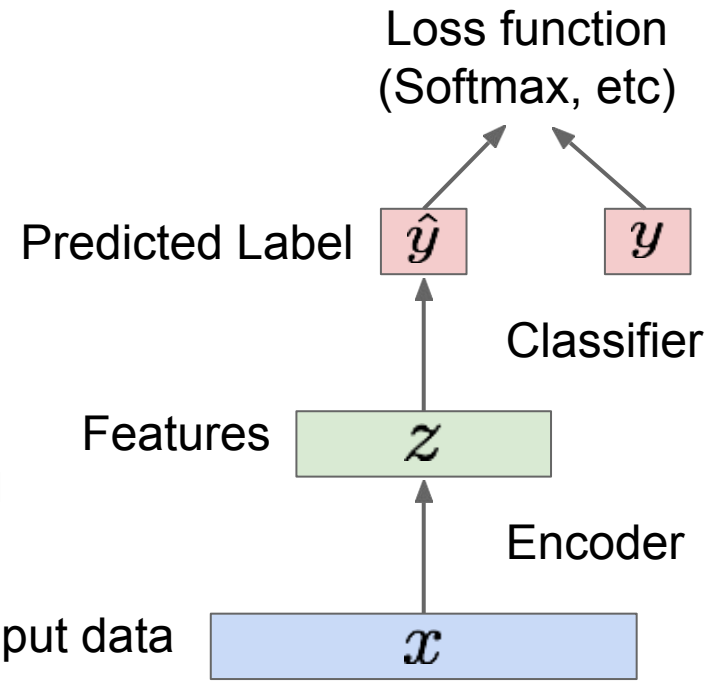
Encoder: 4-layer conv

Input data

Some background first: AutoEncoders



Some background first: AutoEncoders



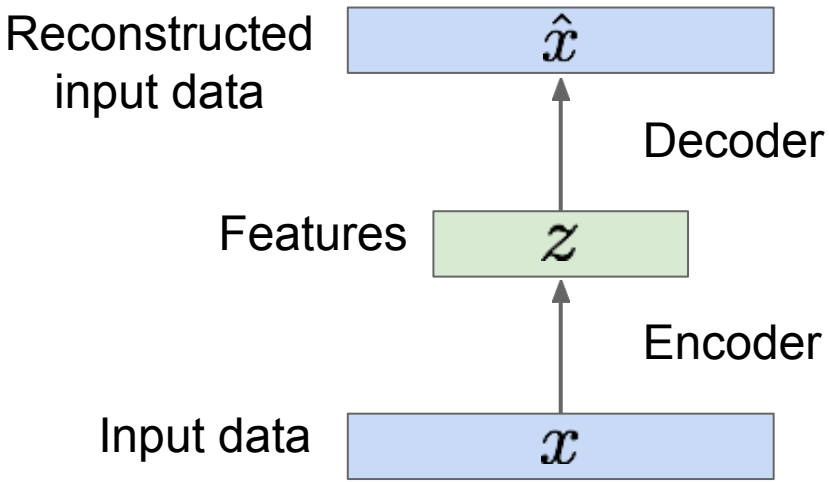
Encoder can be used to initialize a **supervised** model

Fine-tune encoder jointly with classifier

Train for final task (sometimes with small data)

bird plane
dog deer truck

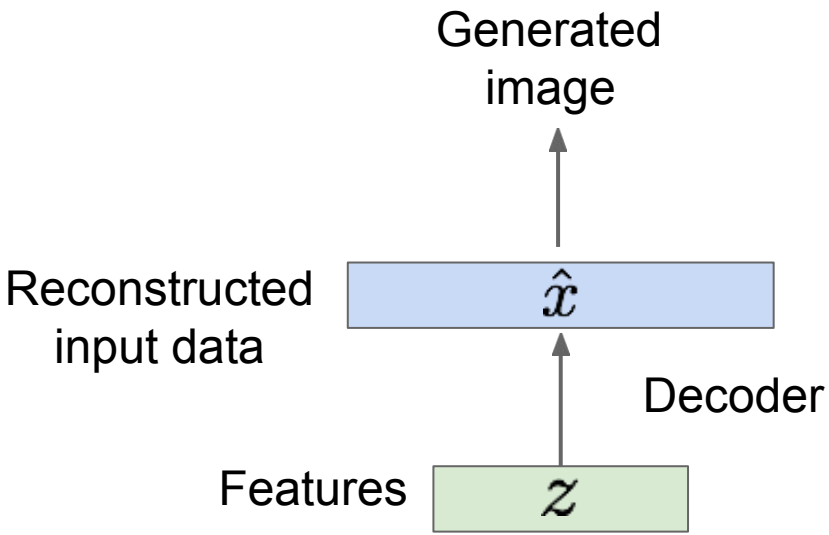
Some background first: AutoEncoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data.

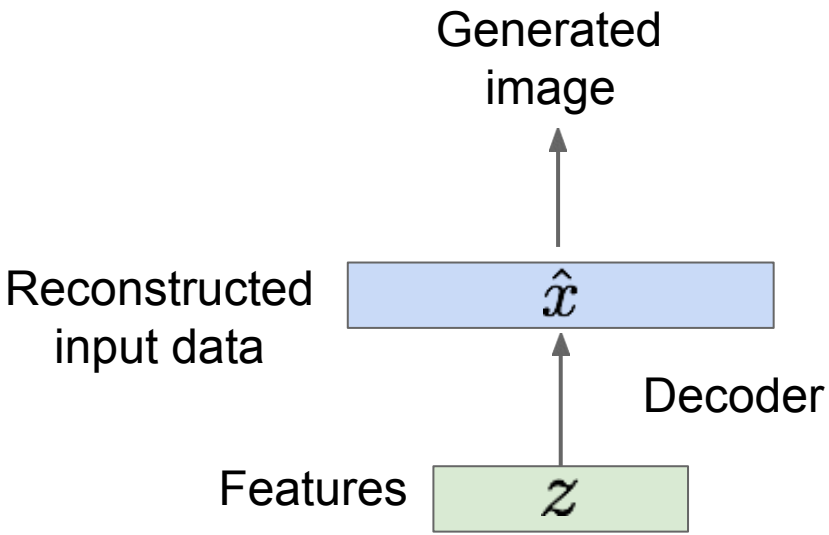
Some background first: AutoEncoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data. Can we **generate new images** from an autoencoder?

Some background first: AutoEncoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data. Can we **generate new images** from an autoencoder?

Problem: Generating new z is not any easier than generating new x

Solution: What if we force all z to come from a known distribution?

Variational AutoEncoders

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

1. Learn latent features z from raw data
2. Sample from the model to generate new data

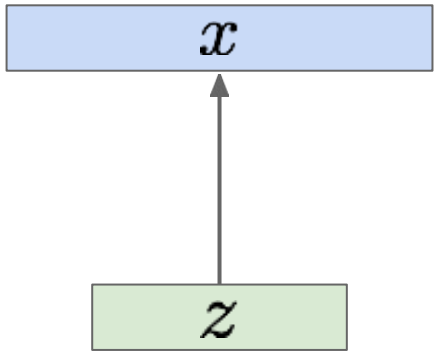
Variational AutoEncoders

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from underlying unobserved (latent) representation **z**

Sample from true conditional $p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior $p_{\theta^*}(z)$



Intuition. **x** is an image, **z** is latent factors used to generate **x:** attributes, orientation, etc.

Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

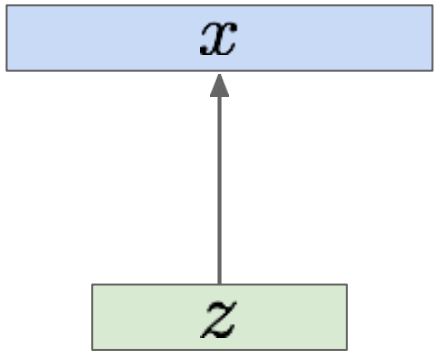
Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

How should we represent this model?

Sample from true conditional $p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior $p_{\theta^*}(z)$



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

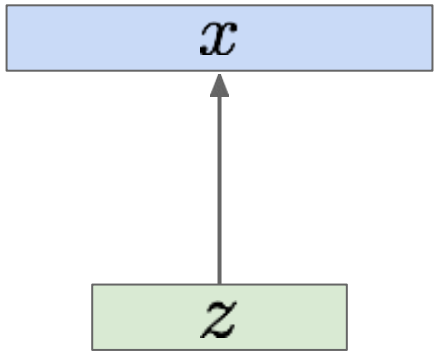
We want to estimate the true parameters θ^* of this generative model.

How should we represent this model?

Choose prior p(z) to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g. pose, how much smile.

Sample from true conditional $p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior $p_{\theta^*}(z)$



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

How should we represent this model?

Choose prior p(z) to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g. pose, how much smile.

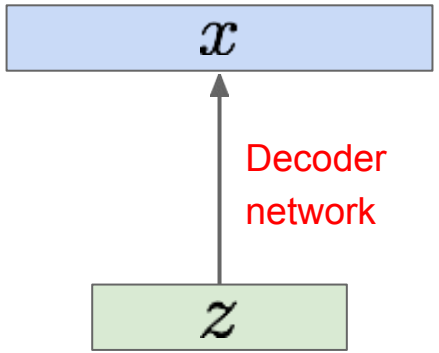
Conditional p(x|z) is complex (generates image) => represent with neural network

Sample from true conditional

$p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior

$p_{\theta^*}(z)$



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

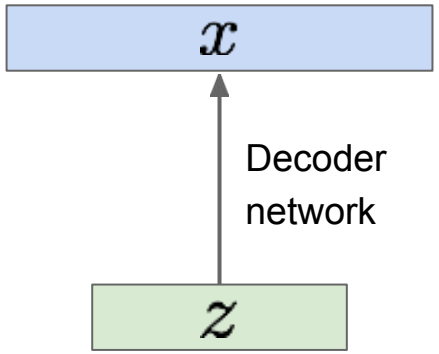
We want to estimate the true parameters θ^* of this generative model.

How to train the model?

Learn model parameters to maximize likelihood of training data

Sample from true conditional $p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior $p_{\theta^*}(z)$



$$p_\theta(x) = \int p_\theta(x,z)dz = \int p_\theta(x|z)p_\theta(z)dz$$

Now with latent z

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

How to train the model?

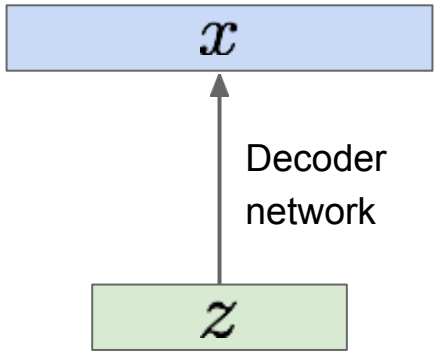
Learn model parameters to maximize likelihood of training data

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

Q: What is the problem with this?

Sample from true conditional $p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior $p_{\theta^*}(z)$



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders Intractability

Data likelihood: $p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$

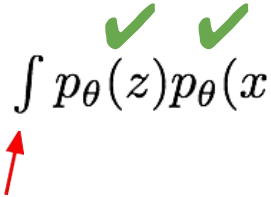
✔ ✔

Simple Gaussian prior

Decoder neural network

Variational AutoEncoders Intractability

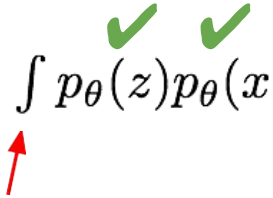
Data likelihood: $p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$



Intractable to compute
p(x|z) for every z!

Variational AutoEncoders Intractability

Data likelihood: $p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$



Intractable to compute
p(x|z) for every z!

Solution: In addition to decoder network modeling $p_\theta(x|z)$, define additional encoder network $q_\phi(z|x)$ that approximates $p_\theta(z|x)$

Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize

Variational AutoEncoders Intractability

Data likelihood: $p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$ ✔ ✔

Intractable to compute p(x|z) for every z!

If we can ensure that $q_\phi(z \mid x) \approx p_\theta(z \mid x)$,

then we can approximate

$$p_\theta(x) \approx \frac{p_\theta(x \mid z) p(z)}{q_\phi(z \mid x)}$$

Idea: Jointly train both encoder and decoder

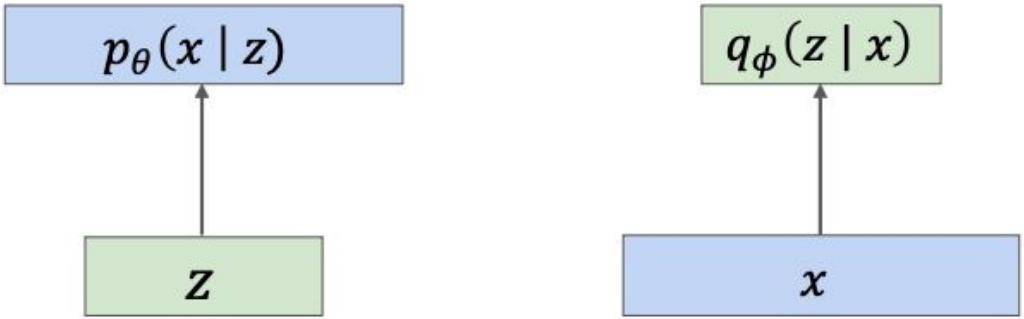
Solution: In addition to decoder network modeling $p_\theta(x|z)$, define additional encoder network $q_\phi(z|x)$ that approximates $p_\theta(z|x)$

Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize

Variational AutoEncoders

Decoder Network:
Input latent code z,
Output distribution over data x

Encoder Network:
Input data x,
Output distribution over latent codes z



If we can ensure that
$q_\phi(z \mid x) \approx p_\theta(z \mid x)$,

then we can approximate

$$p_\theta(x) \approx \frac{p_\theta(x \mid z)p(z)}{q_\phi(z \mid x)}$$

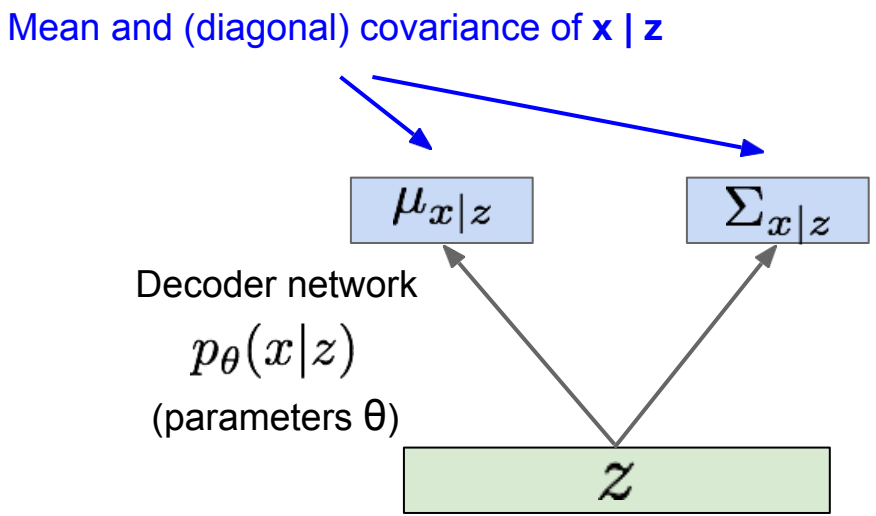
Idea: Jointly train both encoder and decoder

Aside: How to output probability distributions from neural networks?

Network outputs mean (and std) of a (diagonal) distribution

Variational AutoEncoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic

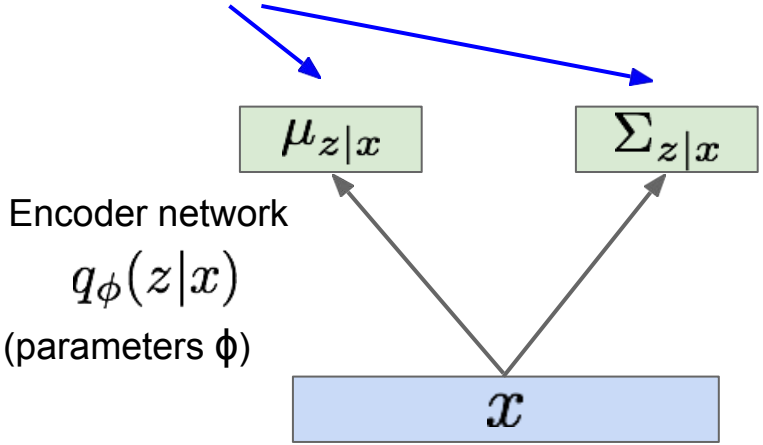


Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

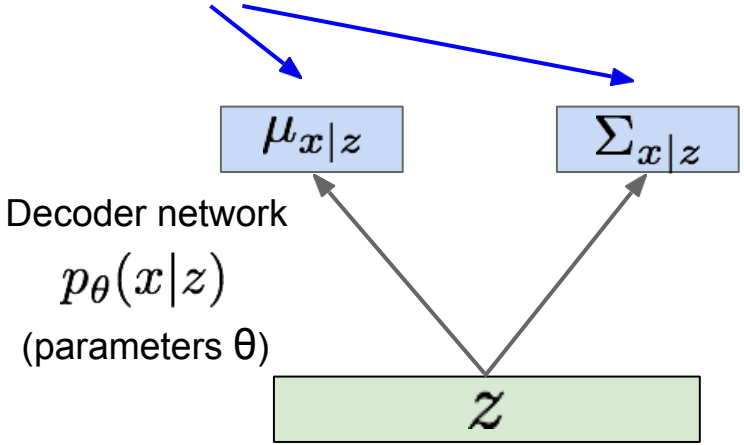
Variational AutoEncoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic

Mean and (diagonal) covariance of **z | x**



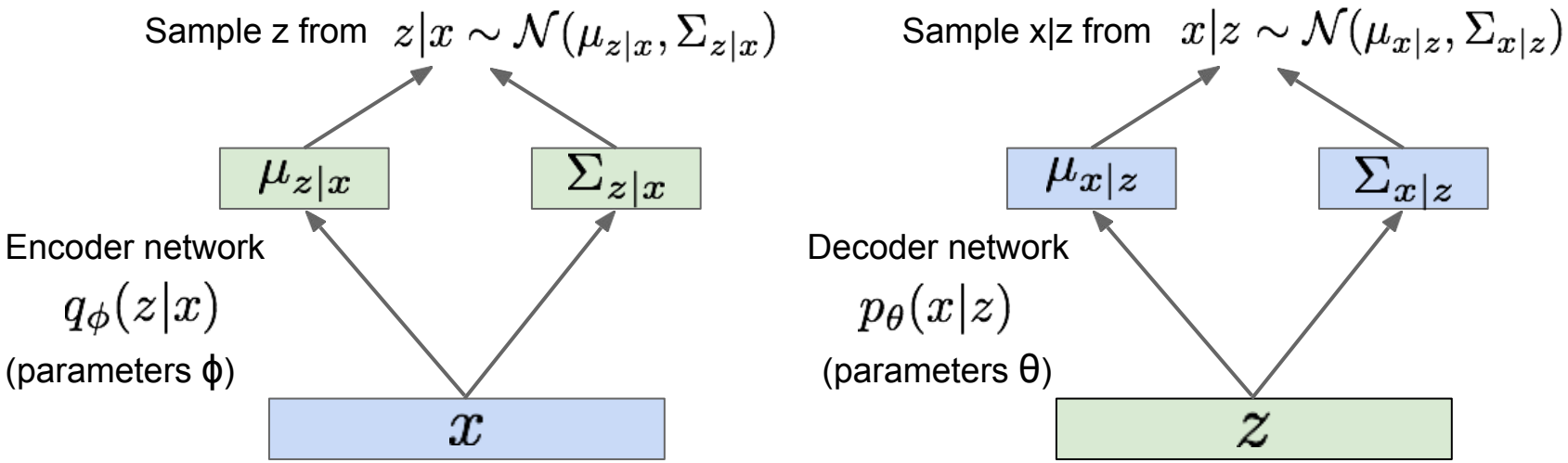
Mean and (diagonal) covariance of **x | z**



Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders

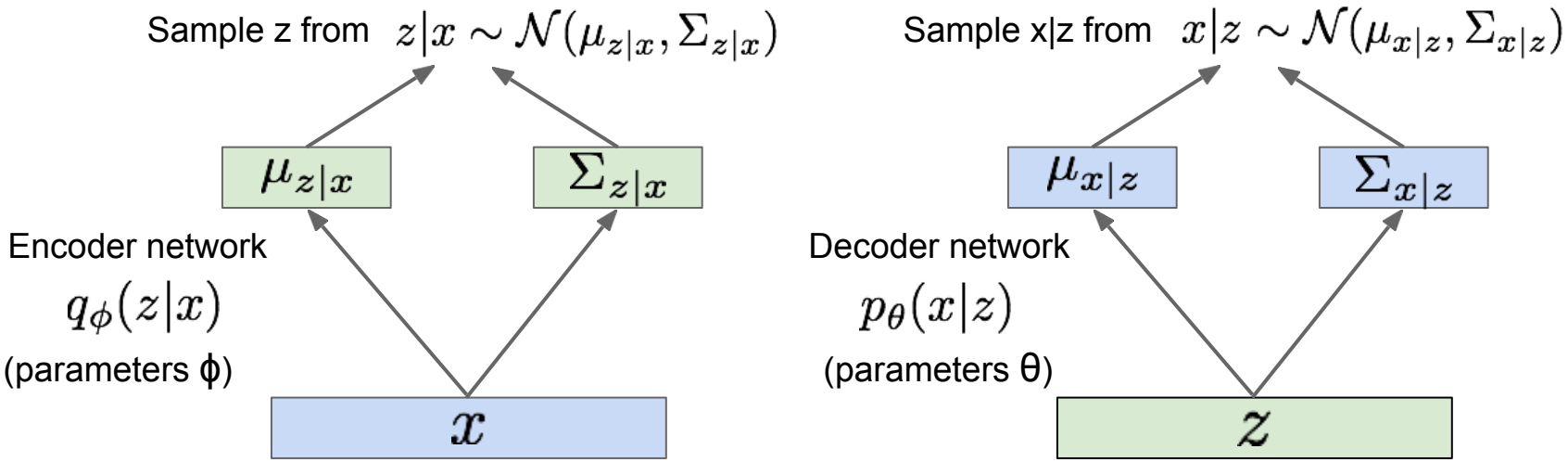
Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic



Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic



$$p_\theta(x) = \frac{p_\theta(x \mid z)p_\theta(z)}{p_\theta(z \mid x)} \approx \frac{p_\theta(x \mid z)p_\theta(z)}{q_\phi(z \mid x)}$$

Use **encoder** to compute $q_\phi(z \mid x) \approx p_\theta(z \mid x)$

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

Taking expectation wrt. z
(using encoder network) will
come in handy later

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad (\text{Bayes' Rule})$$

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad (\text{Bayes' Rule})$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad (\text{Multiply by constant})$$

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z)) + D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))$$

The expectation wrt. z (using encoder network) let us write nice KL terms

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z)) + D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))$$

Decoder network gives $p_\theta(x|z)$.
z is sampled from the encoder $q_\Phi(z|x)$ and the decoder should reconstruct x.
Computed in closed form for Gaussians.

This KL term (between Gaussians for encoder and z prior) has nice closed-form solution!

$p_\theta(z|x)$ intractable (saw earlier), can't compute this KL term :(
But we know KL divergence always >= 0.

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

$$= \underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))}_{\geq 0}$$

Tractable lower bound which we can take gradient of and optimize! ($p_\theta(x|z)$ differentiable, KL term differentiable)

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

$$= \underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)},\theta,\phi)} + \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))}_{>0}$$

$$\log p_\theta(x^{(i)}) \geq \mathcal{L}(x^{(i)},\theta,\phi)$$

Variational lower bound ("ELBO")

$$\theta^*, \phi^* = \arg\max_{\theta,\phi} \sum_{i=1}^{N} \mathcal{L}(x^{(i)},\theta,\phi)$$

Training: Maximize lower bound

Variational AutoEncoders

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

$$= \underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)},\theta,\phi)} + \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))}_{>0}$$

Reconstruct the input data

Make approximate posterior distribution close to prior

$$\log p_\theta(x^{(i)}) \geq \mathcal{L}(x^{(i)},\theta,\phi)$$

Evidence Lower Bound ("ELBo")

$$\theta^*, \phi^* = \arg\max_{\theta,\phi} \sum_{i=1}^{N} \mathcal{L}(x^{(i)},\theta,\phi)$$

Training: Maximize lower bound

Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

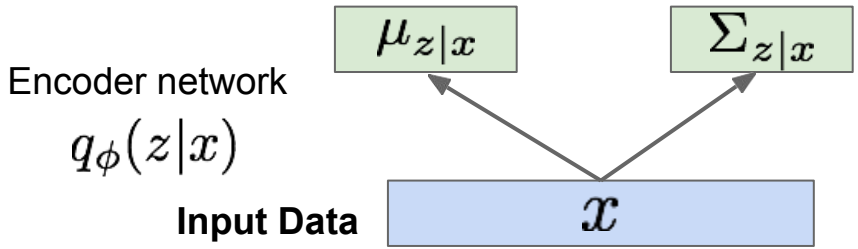
Let's look at computing the bound (forward pass) for a given minibatch of input data

Input Data x

Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

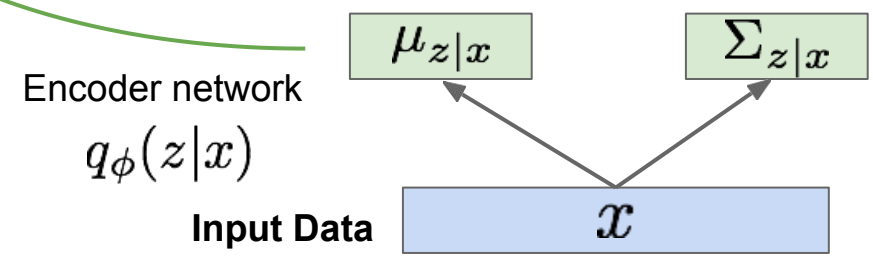


Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Make approximate posterior distribution close to prior

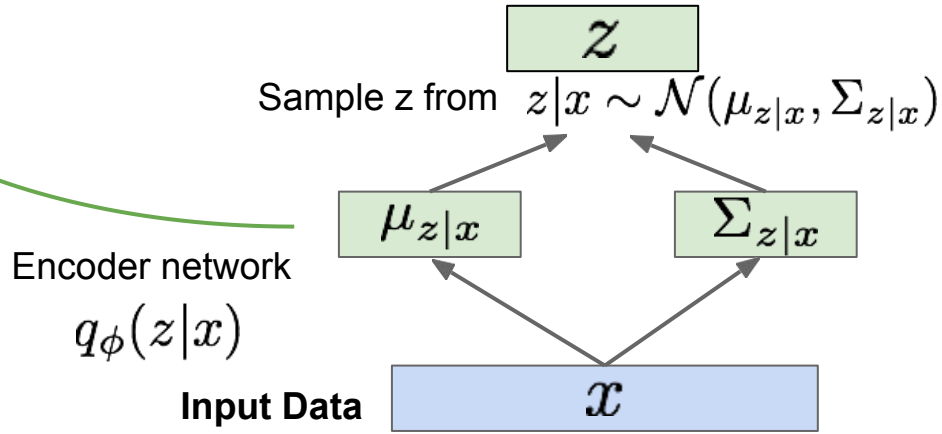


Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

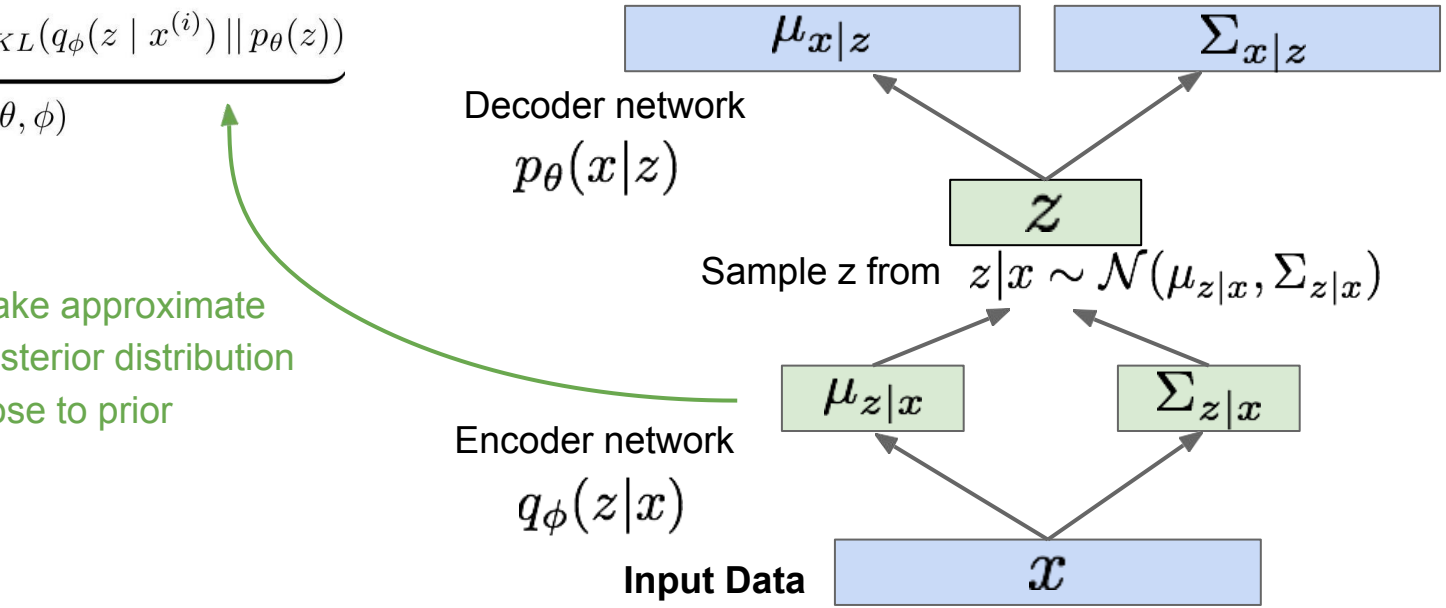
Make approximate posterior distribution close to prior



Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$



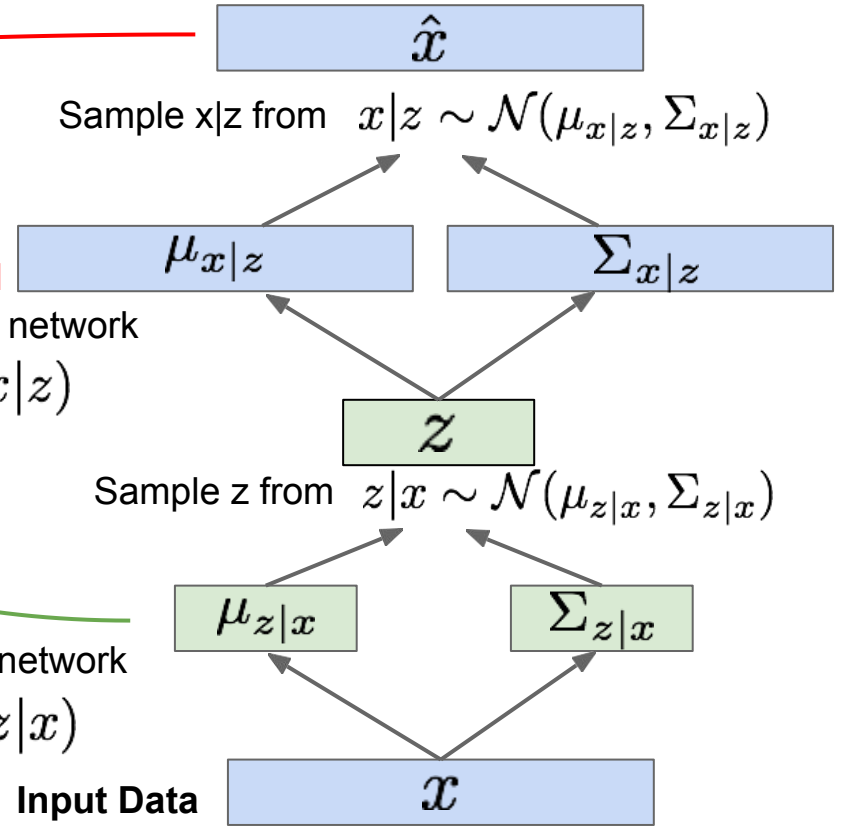
Variational AutoEncoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Maximize likelihood of original input being reconstructed

Make approximate posterior distribution close to prior



Variational AutoEncoders

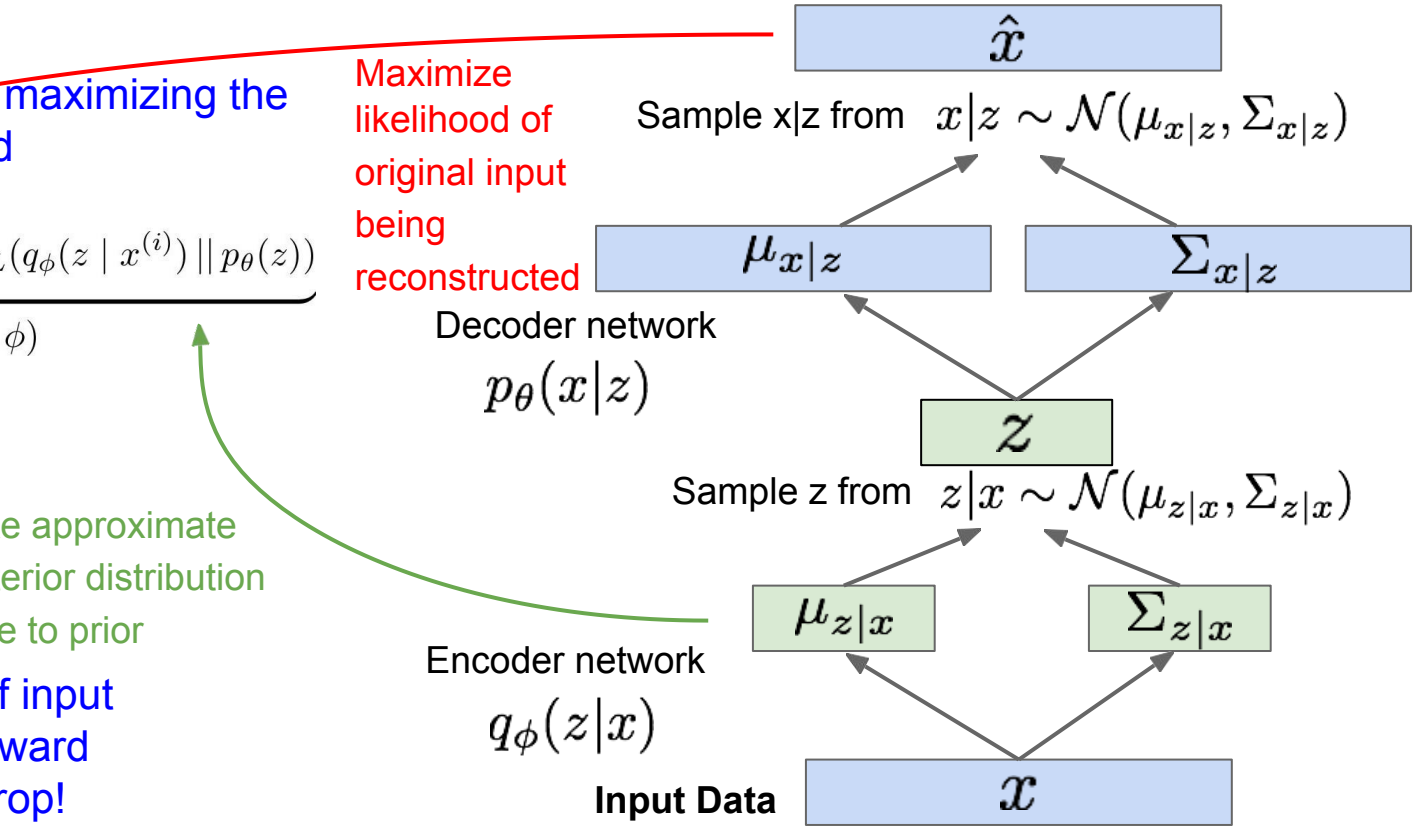
Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Maximize likelihood of original input being reconstructed

Make approximate posterior distribution close to prior

For every minibatch of input data: compute this forward pass, and then backprop!



Training

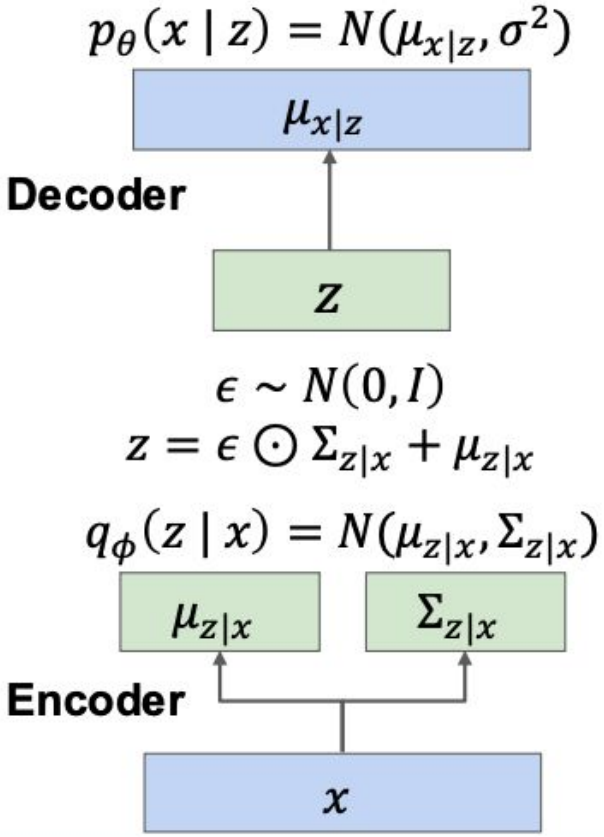
Train by maximizing the **variational lower bound**

$$E_{z\sim q_\phi(z|x)}[log\ p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$

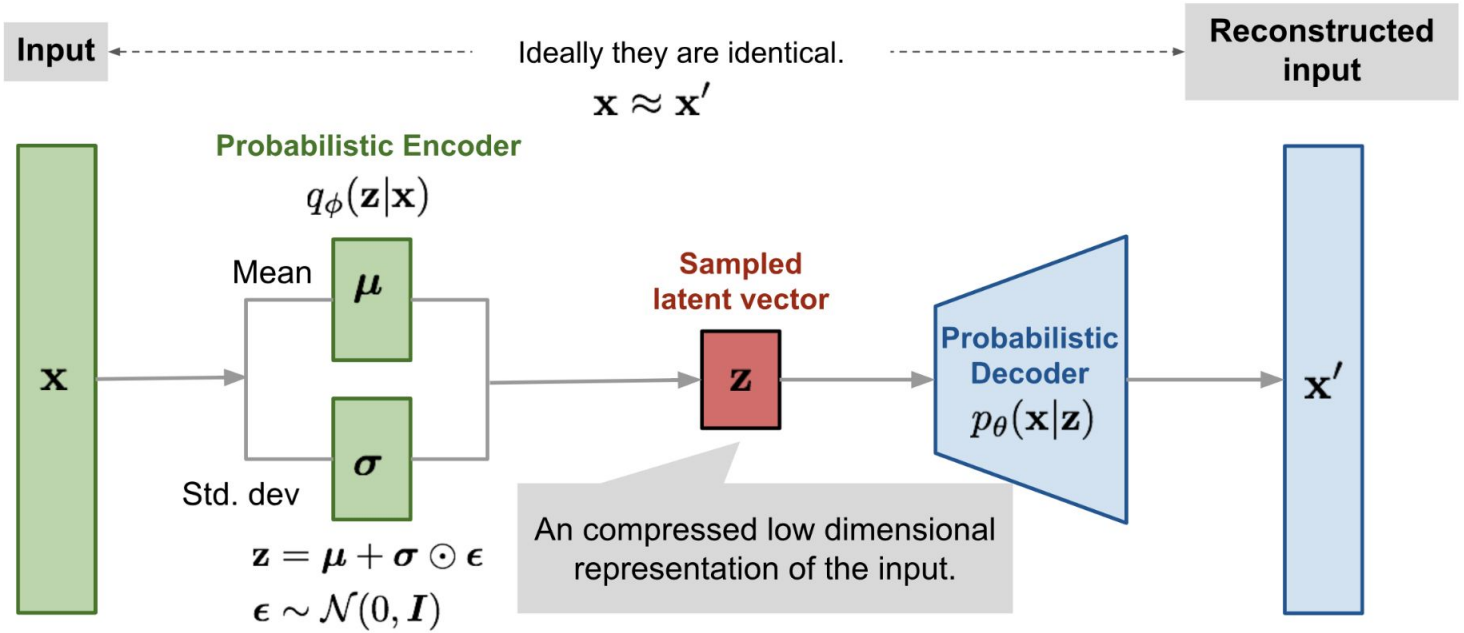
The loss terms fight against each other!

Reconstruction loss wants $\Sigma_{z|x} = 0$ and $\mu_{z|x}$ to be unique for each x, so decoder can deterministically reconstruct x

Prior loss wants $\Sigma_{z|x} = \mathbf{I}$ and $\mu_{z|x} = 0$ so encoder output is always a unit Gaussian

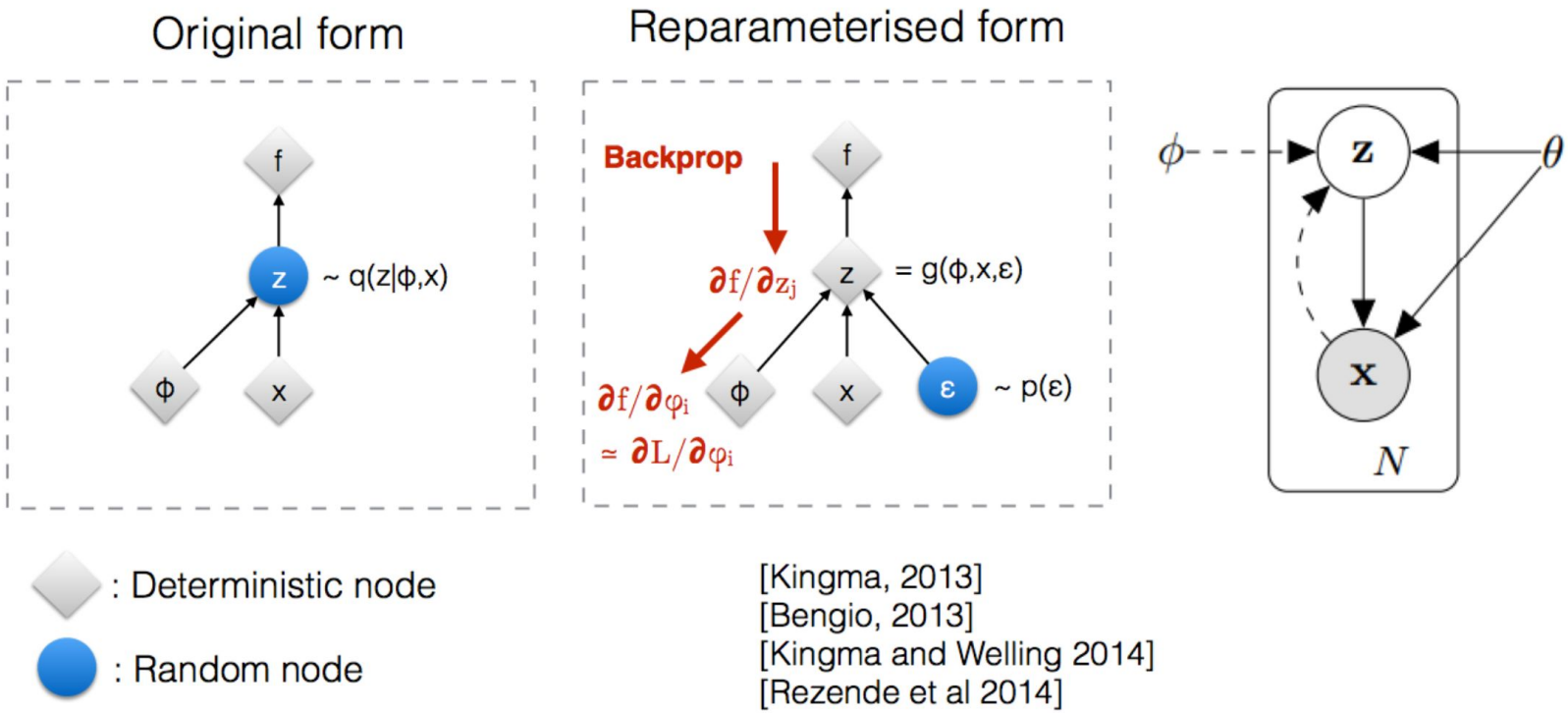


Variational AutoEncoders: Generating Data



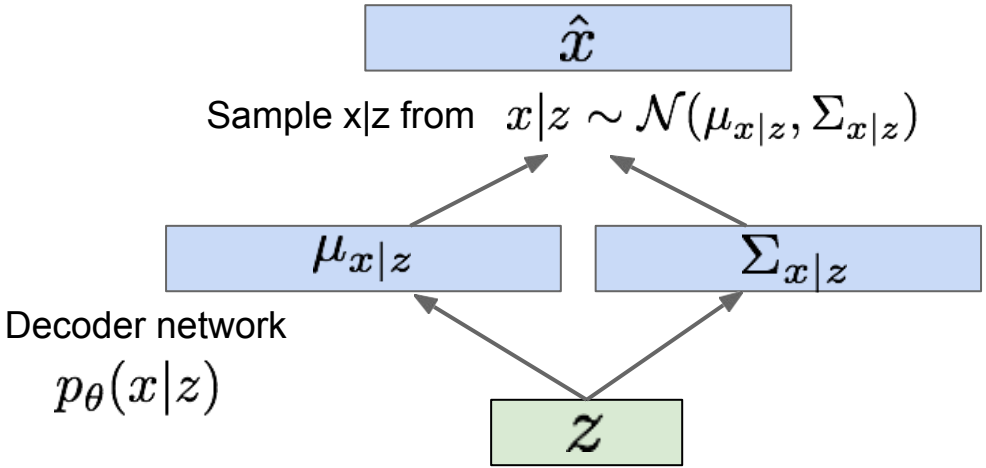
https://lilianweng.github.io/lil-log/2018/08/12/from-autoencoder-to-beta-vae.html

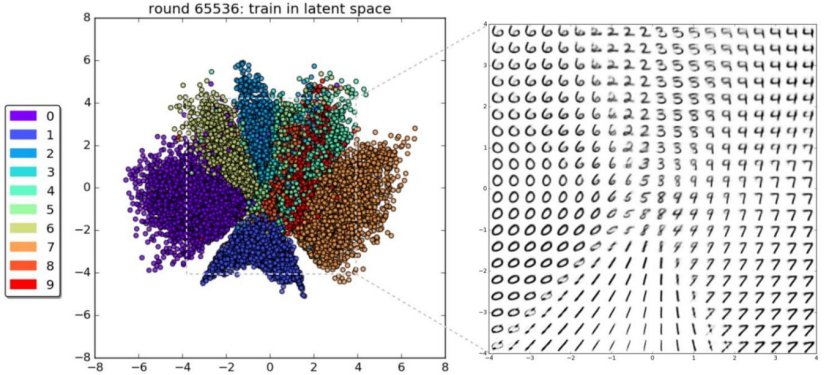
Re-parametrization Trick: Graphically



Variational AutoEncoders: Generating Data

Use decoder network. Now sample z from prior!

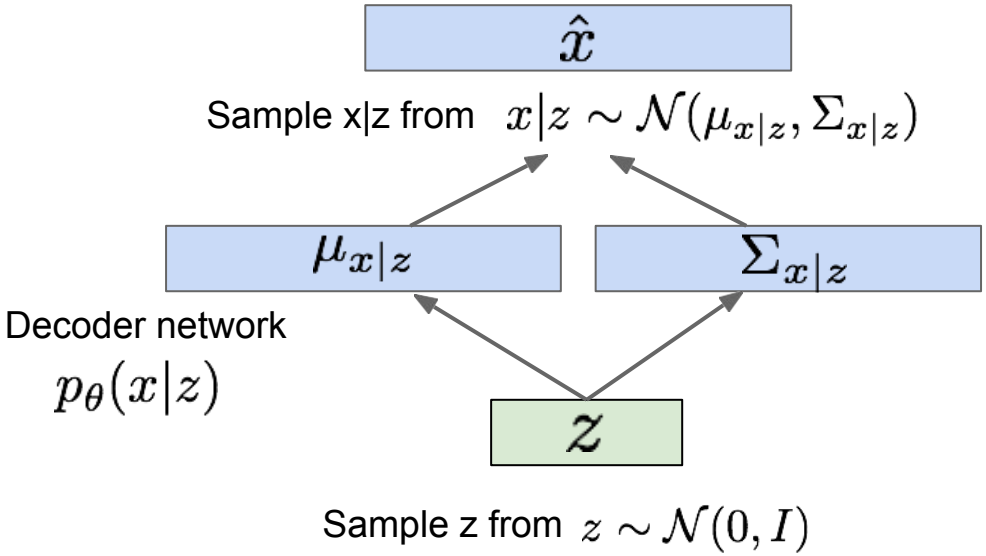


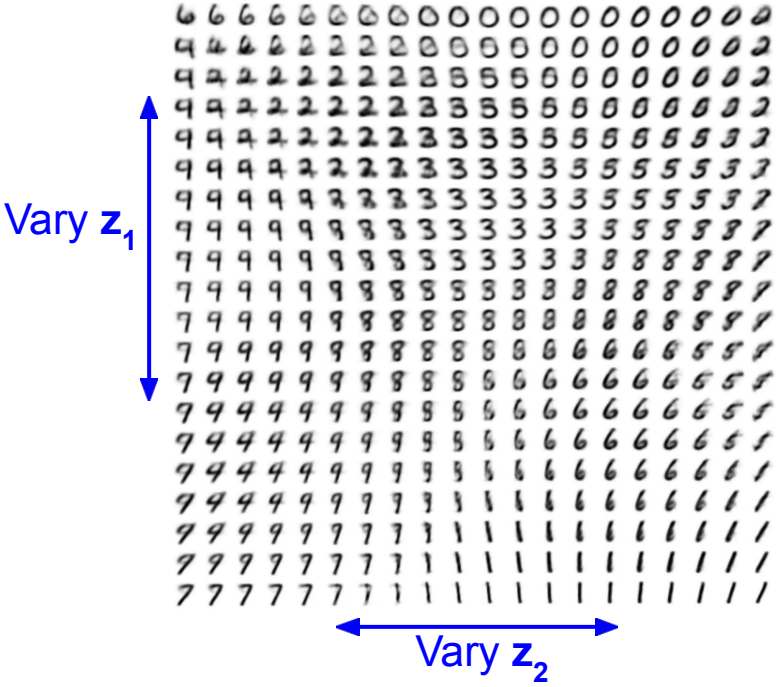


Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders: Generating Data

Use decoder network. Now sample z from prior! Data manifold for 2-d **z**



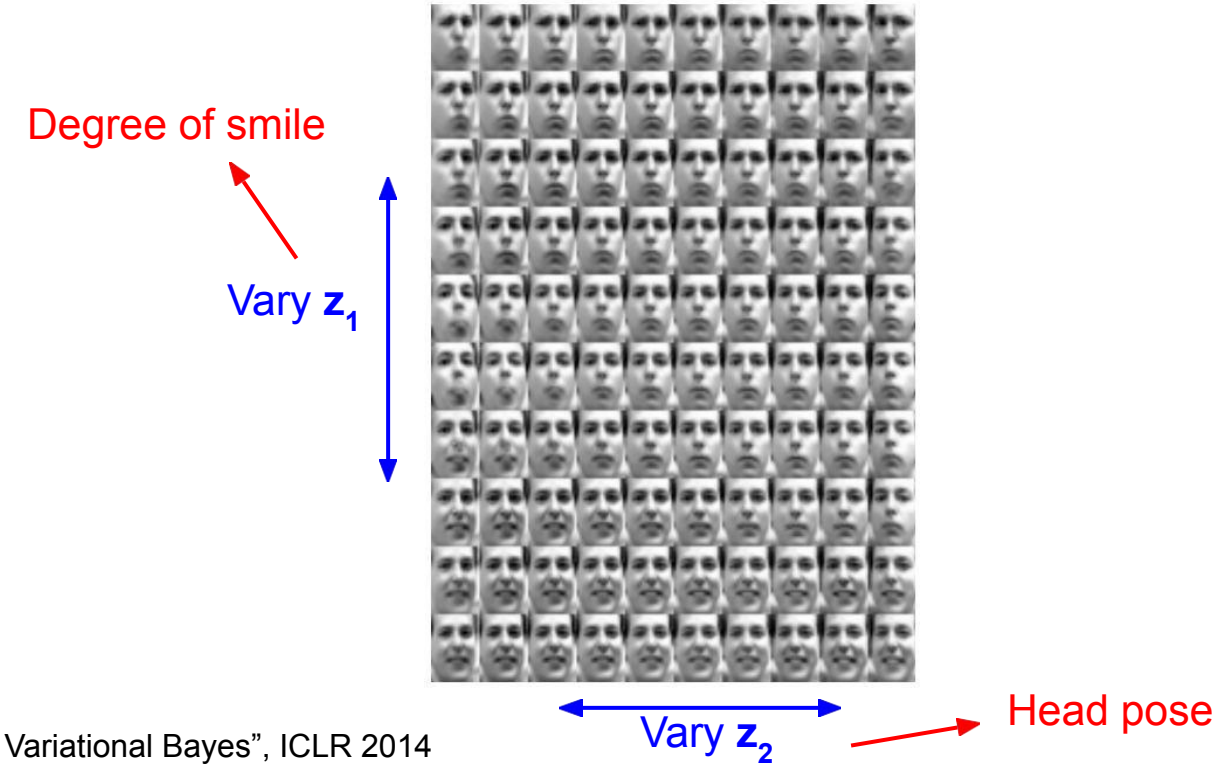


Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders: Generating Data

Diagonal prior on **z** => independent latent variables

Different dimensions of **z** encode interpretable factors of variation



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders: Generating Data

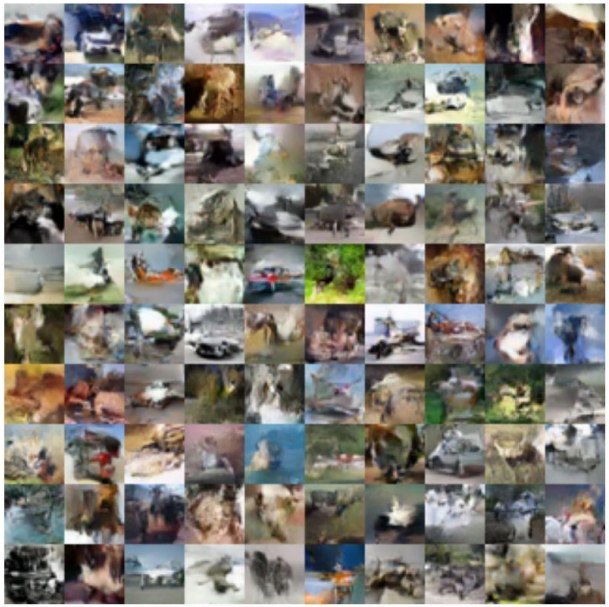
Diagonal prior on **z** => independent latent variables

Different dimensions of **z** encode interpretable factors of variation

Also good feature representation that can be computed using $q_\phi(z|x)$!

Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders: Generating Data

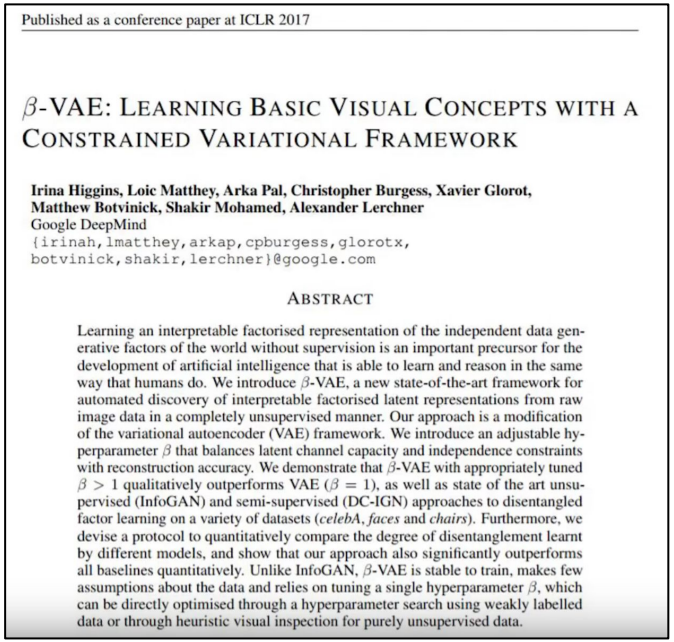


32x32 CIFAR-10

Labeled Faces in the Wild

Disentangled Variational AutoEncoders

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$$



Published as a conference paper at ICLR 2017

β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework

Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, Alexander Lerchner
Google DeepMind
{irinah,lmatthey,arkap,cpburgess,glorotx,
botvinick,shakir,lerchner}@google.com

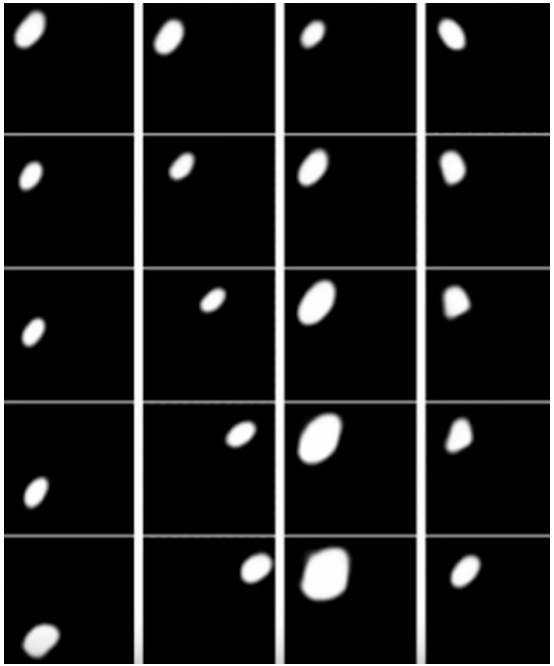
Abstract

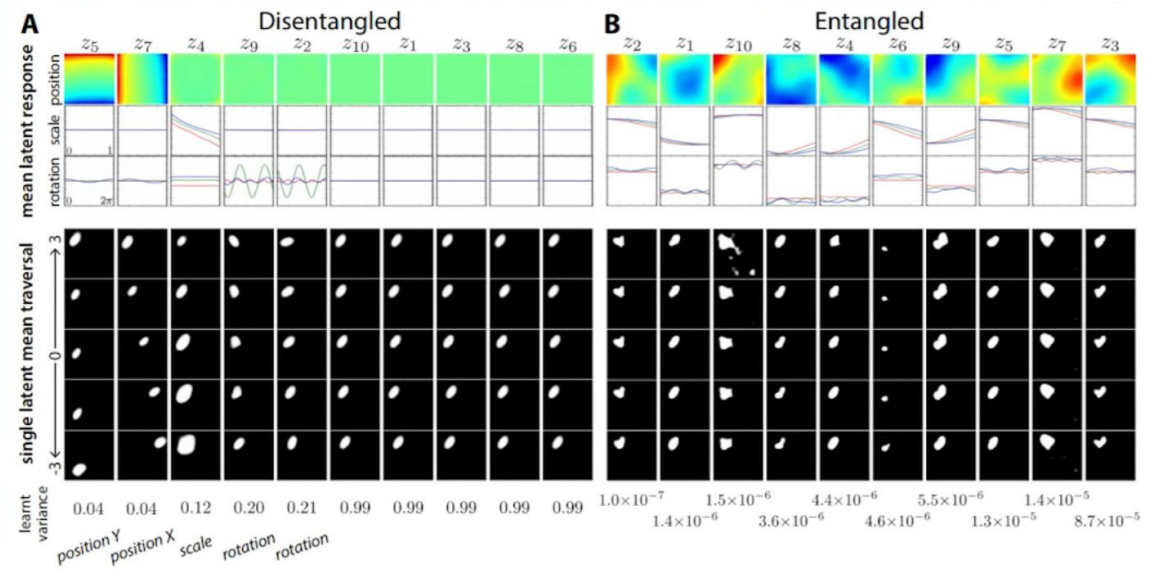
Learning an interpretable factorised representation of the independent data generative factors of the world without supervision is an important precursor for the development of artificial intelligence that is able to learn and reason in the same way that humans do. We introduce β-VAE, a new state-of-the-art framework for automated discovery of interpretable factorised latent representations from raw image data in a completely unsupervised manner. Our approach is a modification of the variational autoencoder (VAE) framework. We introduce an adjustable hyperparameter β that balances latent channel capacity and independence constraints with reconstruction accuracy. We demonstrate that β-VAE with appropriately tuned $\beta > 1$ qualitatively outperforms VAE ($\beta = 1$), as well as state of the art unsupervised (InfoGAN) and semi-supervised (DC-IGN) approaches to disentangled factor learning on a variety of datasets (*celebA*, *faces* and *chairs*). Furthermore, we devise a protocol to quantitatively compare the degree of disentanglement learnt by different models, and show that our approach also significantly outperforms all baselines quantitatively. Unlike InfoGAN, β-VAE is stable to train, makes few assumptions about the data and relies on tuning a single hyperparameter β, which can be directly optimised through a hyperparameter search using weakly labelled data or through heuristic visual inspection for purely unsupervised data.

Slide credit: Fei Fei Li

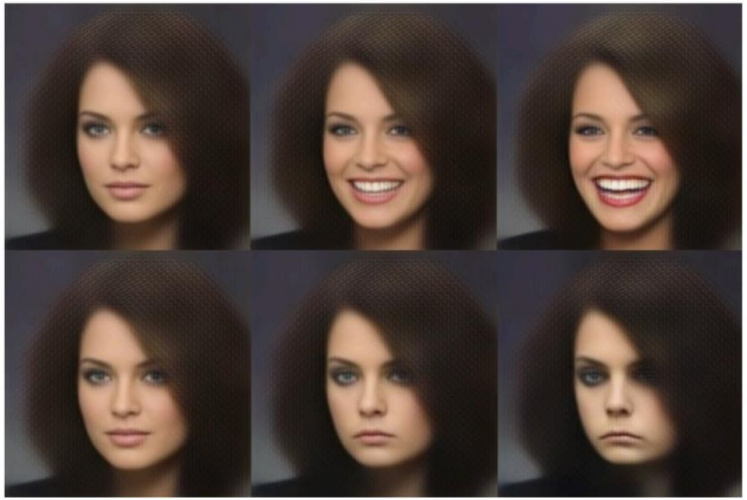
Disentangled Variational AutoEncoders

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$$





Variational AutoEncoders: Generating Data



Smile vector:
mean smiling faces −
mean no-smile faces

Latent space arithmetic

Figure 2.D.3: VAEs can be used for image re-synthesis. In this example by White [2016], an original image (left) is modified in a latent space in the direction of a *smile vector*, producing a range of versions of the original, from smiling to sadness. Notice how changing the image along a single vector in latent space, modifies the image in many subtle and less-subtle ways in pixel space.

Variational AutoEncoders

Probabilistic spin to traditional autoencoders => allows generating data
Defines an intractable density => derive and optimize a (variational) lower bound

Pros:

- Principled approach to generative models
- Allows inference of q(z|x), can be useful feature representation for other tasks

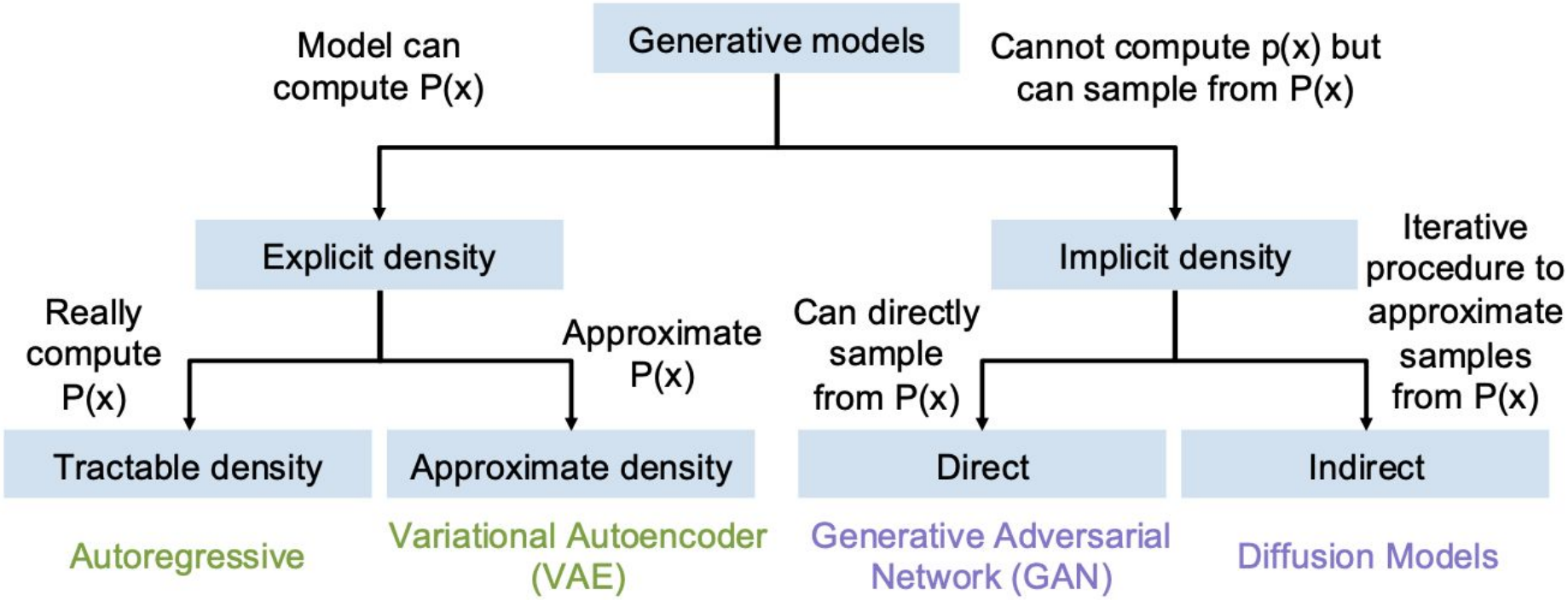
Cons:

- Maximizes lower bound of likelihood: okay, but not as good evaluation as PixelRNN/PixelCNN
- Samples blurrier and lower quality compared to GANs

Active areas of research:

- More flexible approximations, e.g. richer approximate posterior instead of diagonal Gaussian
- Incorporating structure in latent variables

Taxonomy of Generative Models



Slide credit: Justin Johnson

So Far...

Autoregressive models define tractable density function, optimize likelihood of training data:

$$p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i | x_1, ..., x_{i-1})$$

Variational AutoEncoders (VAEs)
Define intractable density function with latent **z**:

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

Cannot optimize directly, derive and optimize lower bound on likelihood instead

What if we give up on explicitly modeling density, and just want ability to sample?
GANs: don't work with any explicit density function! No modeling p(x) .
Instead, take game-theoretic approach: learn to generate from training distribution through 2-player game. This will allow us to sample from p(x) .

Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Problem: Want to sample from complex, high-dimensional training distribution. No direct way to do this!

Solution: Sample from a simple distribution, e.g. random noise. Learn transformation to training distribution.

Q: What can we use to represent this complex transformation?

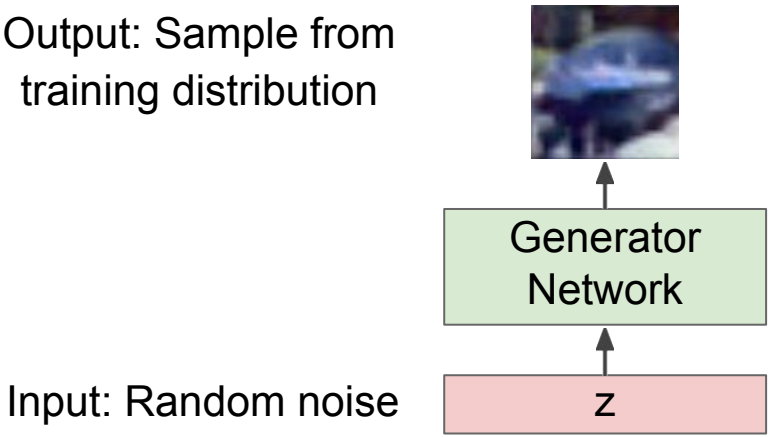
Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Problem: Want to sample from complex, high-dimensional training distribution. No direct way to do this!

Solution: Sample from a simple distribution, e.g. random noise. Learn transformation to training distribution.

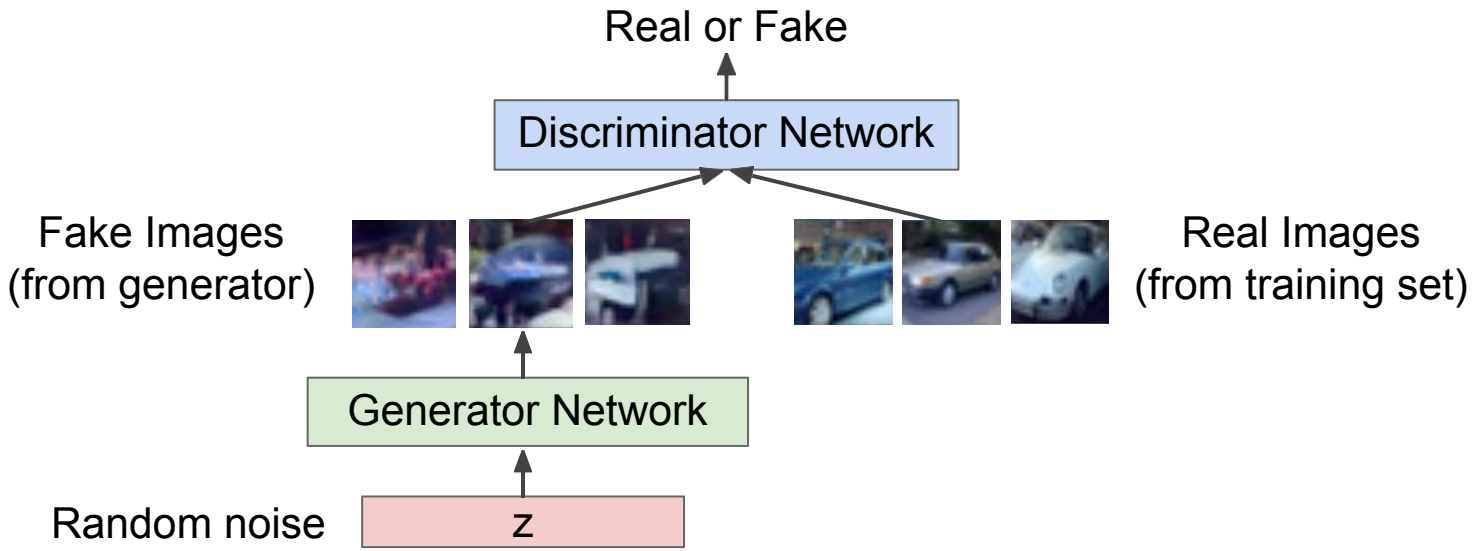
Q: What can we use to represent this complex transformation?



Training GANs: two player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images



Fake and real images copyright Emily Denton et al. 2015. Reproduced with permission.

Training GANs: two player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images

Train jointly in **minimax game**

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Training GANs: two player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images

Train jointly in **minimax game**

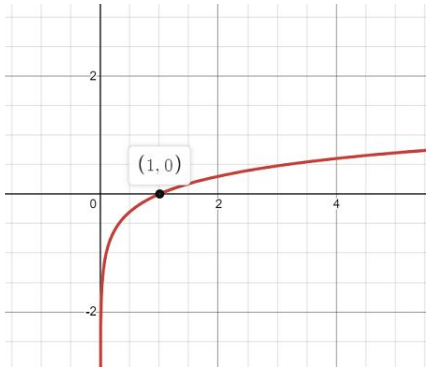
Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data G(z)



Training GANs: two player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data G(z)

- Discriminator (θ_d) wants to **maximize objective** such that D(x) is close to 1 (real) and D(G(z)) is close to 0 (fake)
- Generator (θ_g) wants to **minimize objective** such that D(G(z)) is close to 1 (discriminator is fooled into thinking generated G(z) is real)

Training GANs: two player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

$$\min_{G} \max_{D} \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

$$= \min_{G} \max_{D} V(G, D)$$

We are not minimizing any overall loss! No training curves to look at!

For t in 1, … T:

1. (Update D) $D = D + \alpha_D \frac{\partial V}{\partial D}$
2. (Update G) $G = G - \alpha_G \frac{\partial V}{\partial G}$

Training GANs: two player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

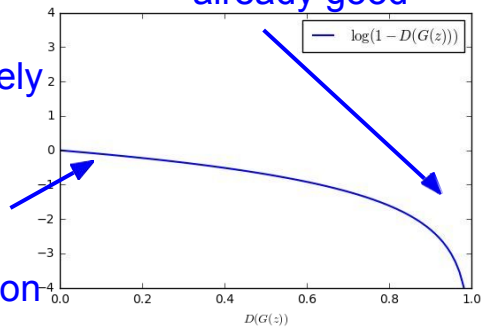
Gradient signal dominated by region where sample is already good

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

In practice, optimizing this generator objective does not work well!

When sample is likely fake, want to learn from it to improve generator. But gradient in this region is relatively flat!



Training GANs: two player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

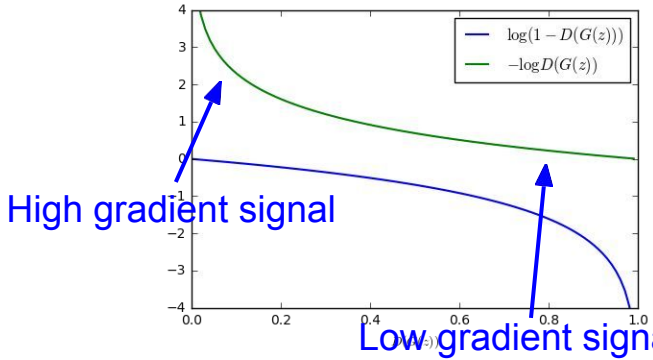
1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Instead: Gradient ascent** on generator,

different objective $\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$

Instead of minimizing likelihood of discriminator being correct, now maximize likelihood of discriminator being wrong.
Same objective of fooling discriminator, but now higher gradient signal for bad samples => works much better! Standard in practice.



Training GANs: two player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Jointly training two networks is challenging, can be unstable. Choosing objectives with better loss landscapes helps training, has been an active area of research.

Alternate between:

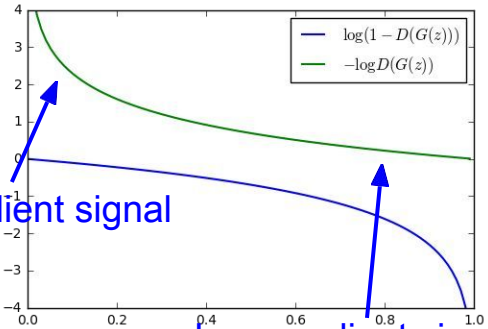
1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Instead: Gradient ascent** on generator,

different objective $\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$

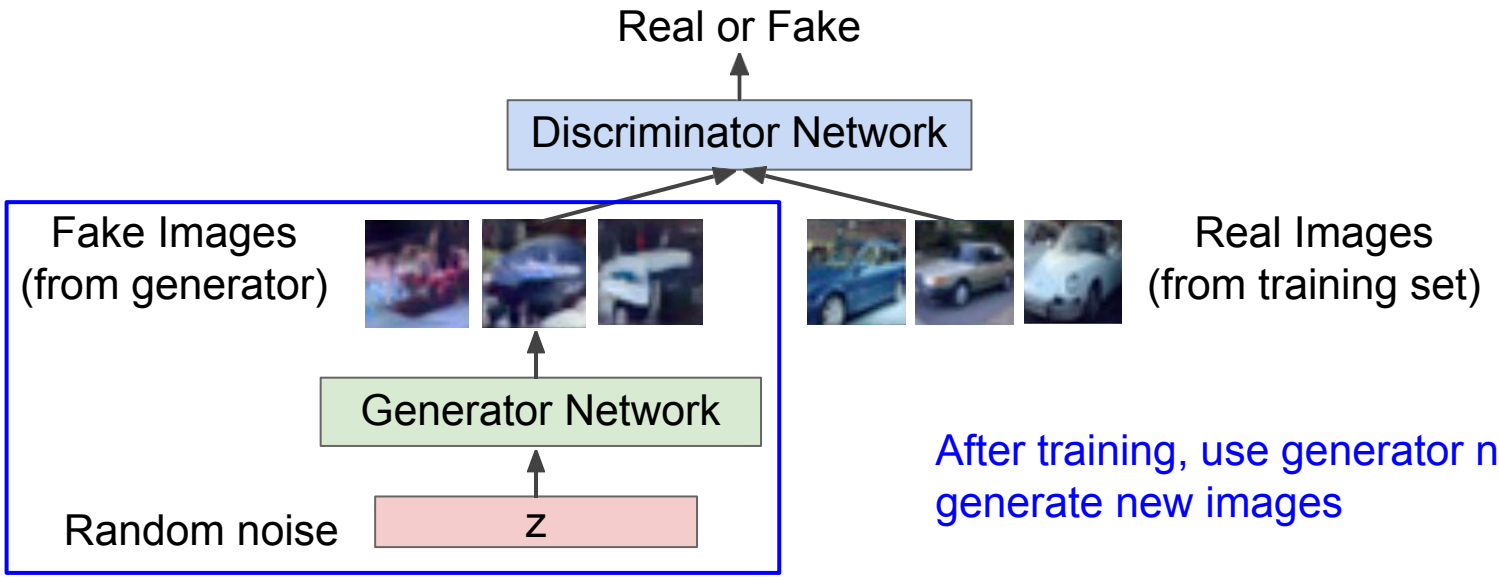
Instead of minimizing likelihood of discriminator being correct, now maximize likelihood of discriminator being wrong.
Same objective of fooling discriminator, but now higher gradient signal for bad samples => works much better! Standard in practice.



Training GANs: two player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images



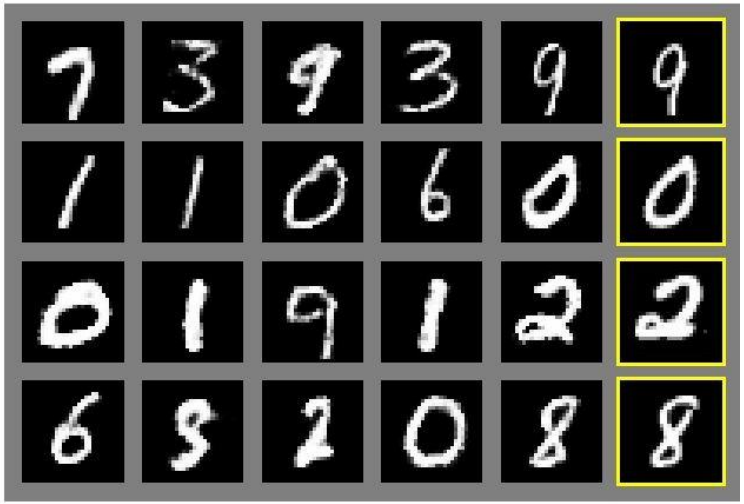
After training, use generator network to generate new images

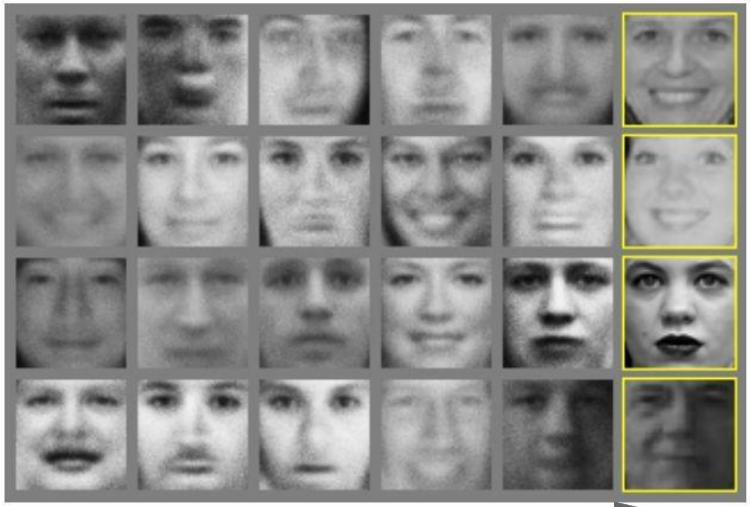
Fake and real images copyright Emily Denton et al. 2015. Reproduced with permission.

Generative Adversarial Nets

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generated samples



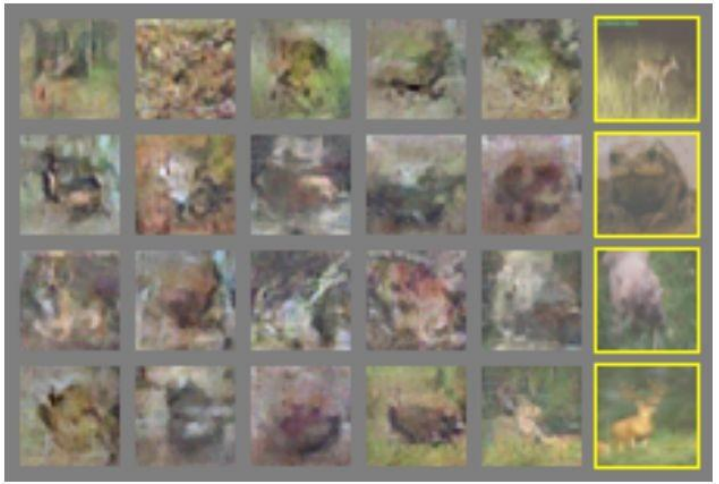


Nearest neighbor from training set

Generative Adversarial Nets

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Generated samples (CIFAR-10)



Nearest neighbor from training set

Generative Adversarial Nets: Architectures

Samples from the model look amazing!

Radford et al, ICLR 2016

Generative Adversarial Nets: Architectures

Interpolating between random points in latent space

Radford et al, ICLR 2016

Generative Adversarial Nets: Architectures

Latent space is **smooth**.

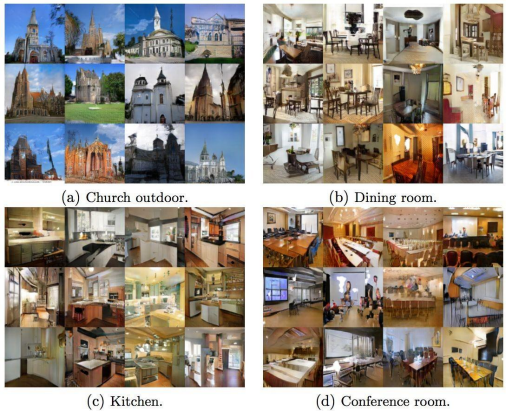
Given latent vectors z_0 and z_1, we can **interpolate** between them:

$$z_t = tz_o + (1-t)z_1$$
$$x_t = G(z_t)$$

The resulting image x_t smoothly interpolate between samples!

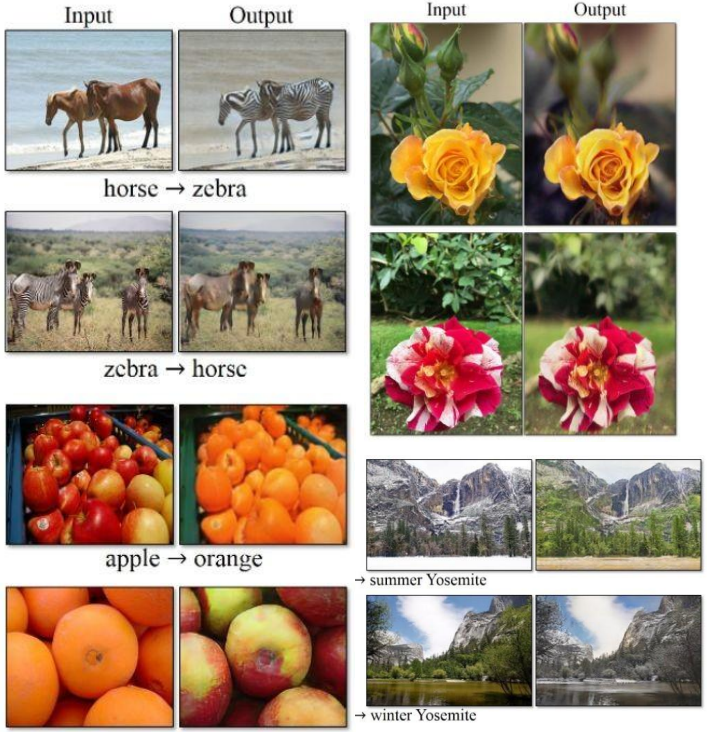
Karras et al, "Alias-Free Generative Adversarial Networks", NeurIPS 2021

2017 : Year of the GAN



LSGAN. Mao et al. 2017.

BEGAN. Bertholet et al. 2017.



CycleGAN. Zhu et al. 2017.

Text -> Image Synthesis

Reed et al. 2017.

Many GAN applications

Pix2pix. Isola 2017. Many examples at https://phillipi.github.io/pix2pix/

GANs

Don’t work with an explicit density function
Take game-theoretic approach: learn to generate from training distribution through 2-player game

Pros:
- Beautiful, ~~state-of-the-art~~ samples!

Cons:
- Trickier / more unstable to train
- Can’t solve inference queries such as p(x), p(z|x)

Active areas of research:
- Better loss functions, more stable training (Wasserstein GAN, LSGAN, many others)
- Conditional GANs, GANs for all kinds of applications

Recap on Generative Models

Generative Models

- Autoregressive — Explicit density model, optimizes exact likelihood, good samples. But inefficient sequential generation.
- Variational Autoencoders (VAE) — Optimize variational lower bound on likelihood. Useful latent representation. But sample quality not the best.
- Generative Adversarial Networks (GANs) — Game-theoretic approach, best samples! But can be tricky and unstable to train.

- combinations
- **diffusion models**: much more powerful!

Overview

- Diffusion Models

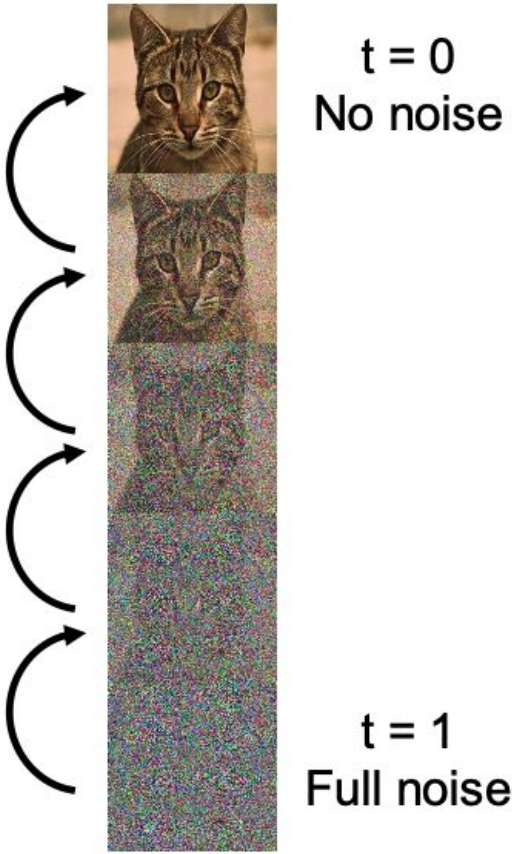
Intuition

Pick a **noise distribution** $z \sim p_{noise}$
(Usually unit Gaussian)

Consider data x corrupted under varying **noise levels** t to give noisy data x_t

Train a neural network to **remove a little bit of noise**: $f_\theta(x_t, t)$

At inference time, sample $x_1 \sim p_{noise}$ and apply f_θ many times in sequence to generate a noiseless sample x_0



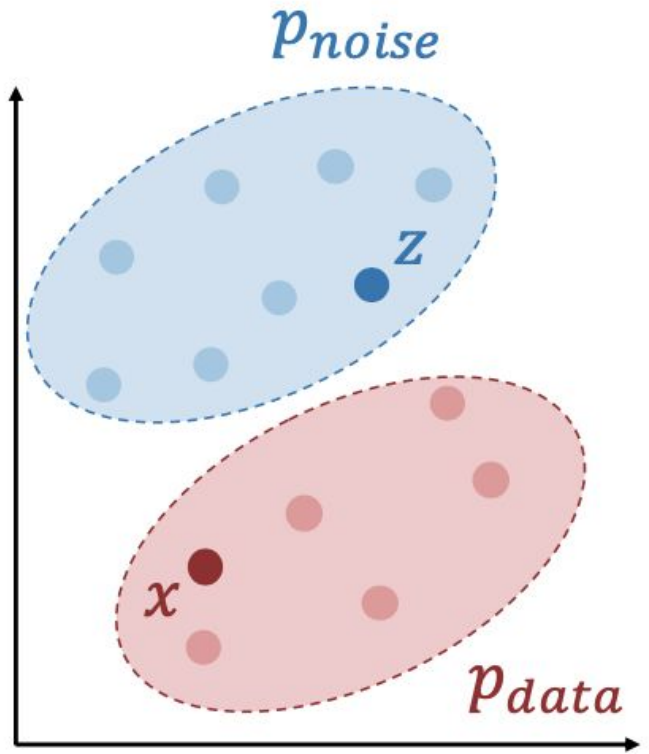
Slide credit: Justin Johnson

Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}

On each training iteration, sample:

$z \sim p_{noise} \quad x \sim p_{data} \quad t \sim Uniform[0, 1]$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}

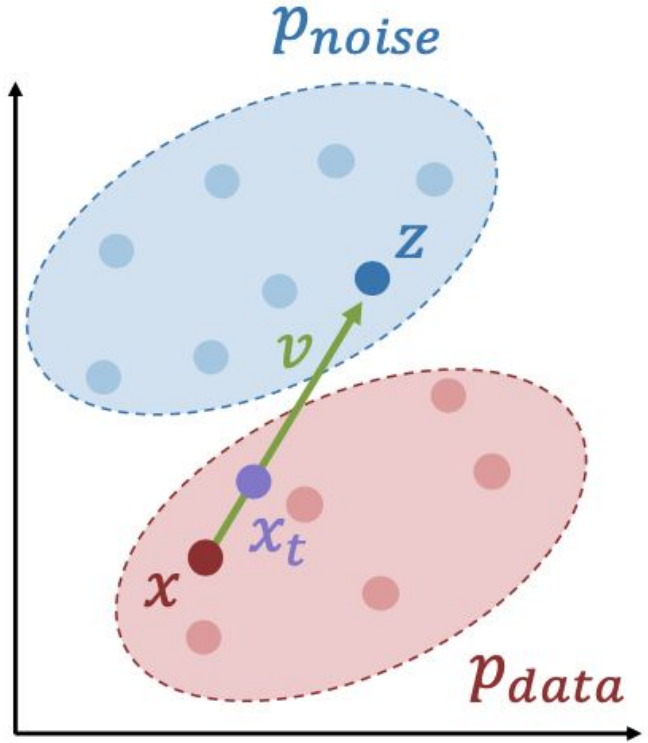
On each training iteration, sample:

$z \sim p_{noise} \quad x \sim p_{data} \quad t \sim Uniform[0, 1]$

Set $x_t = (1 - t)x + tz$, $v = z - x$

Train a neural network to predict v:

$$L = \|f_\theta(x_t, t) - v\|_2^2$$



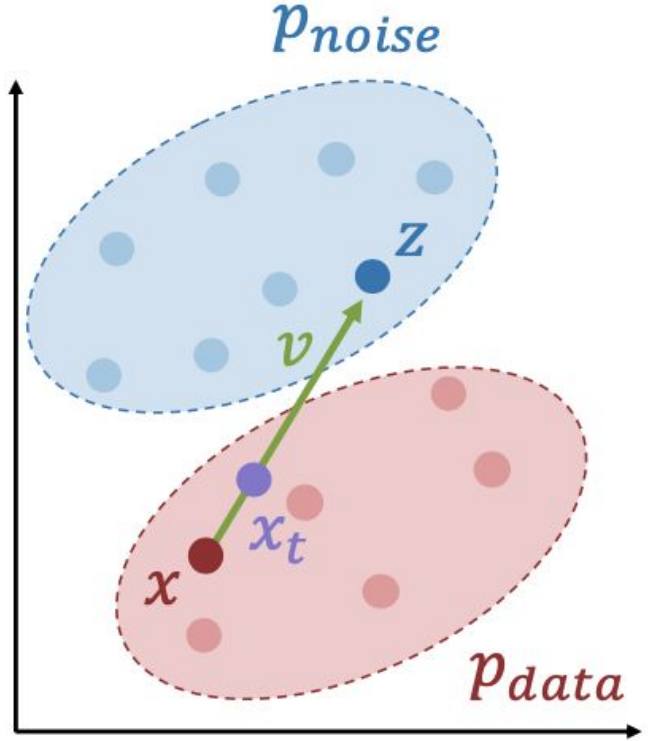
Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Rectified Flow

Core training loop is just a few lines of code!

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```



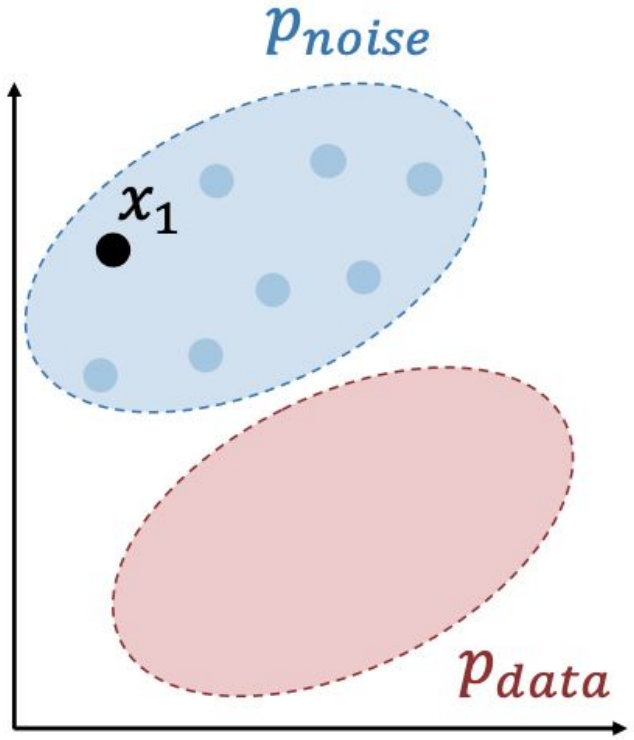
Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

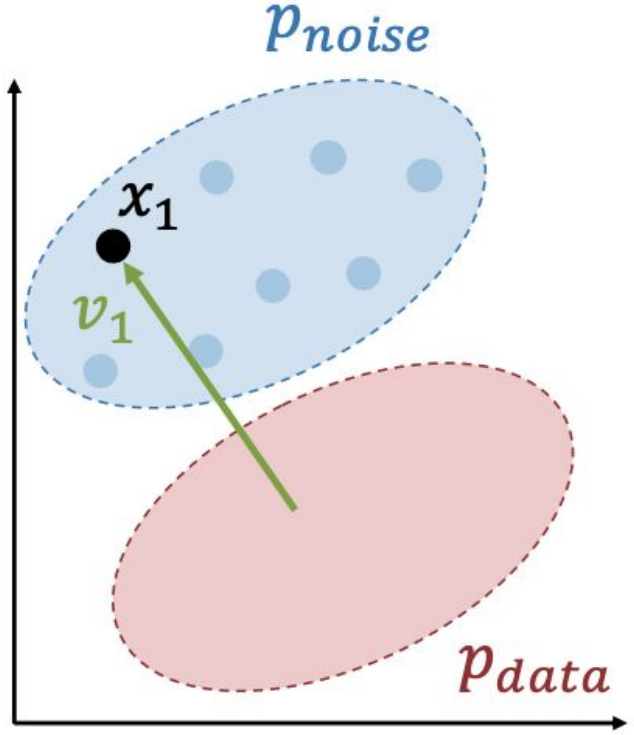
Diffusion Models: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

  Evaluate $v_t = f_\theta(x_t, t)$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Sampling

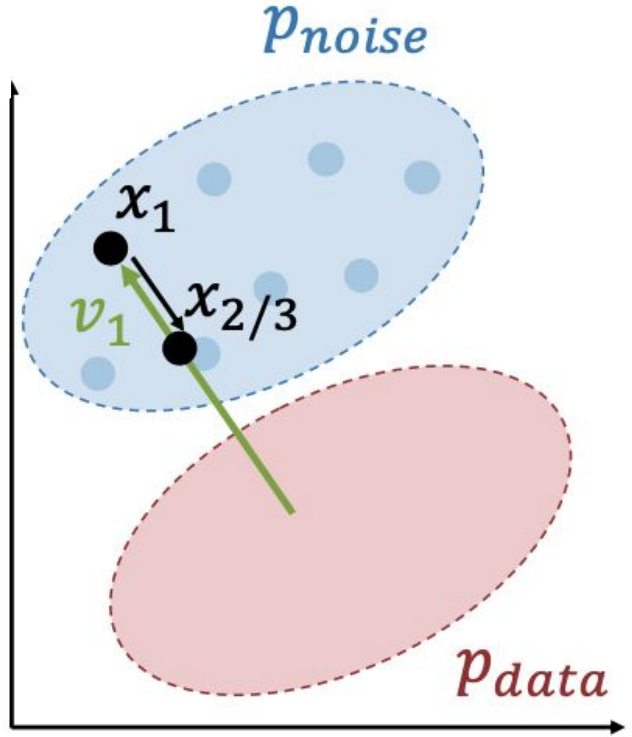
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Sampling

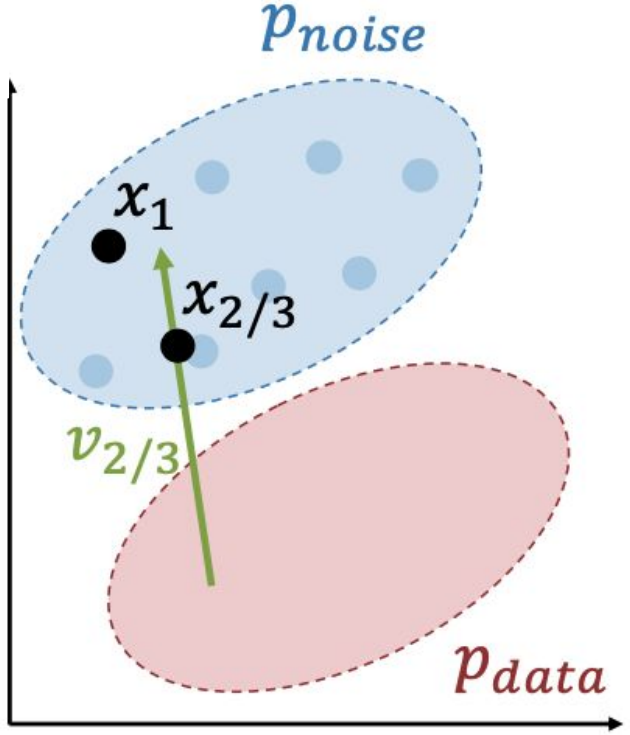
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Sampling

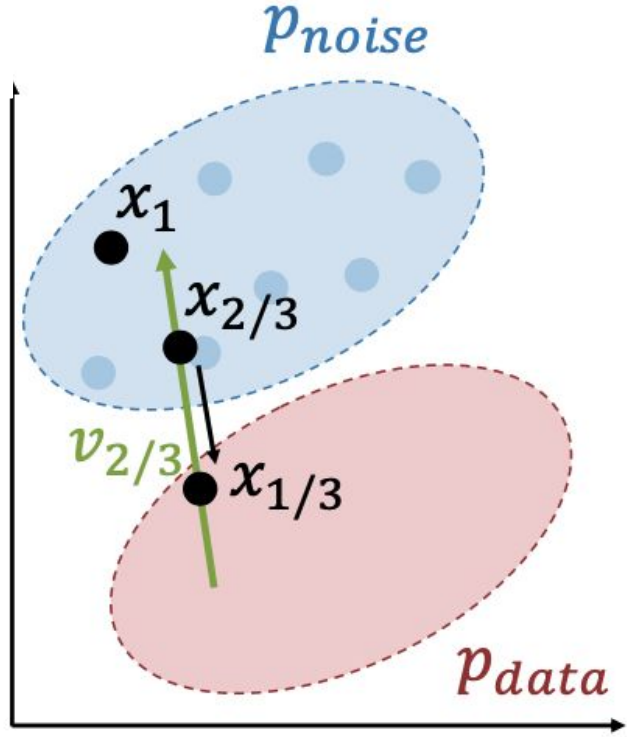
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

  Evaluate $v_t = f_\theta(x_t, t)$

  Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Sampling

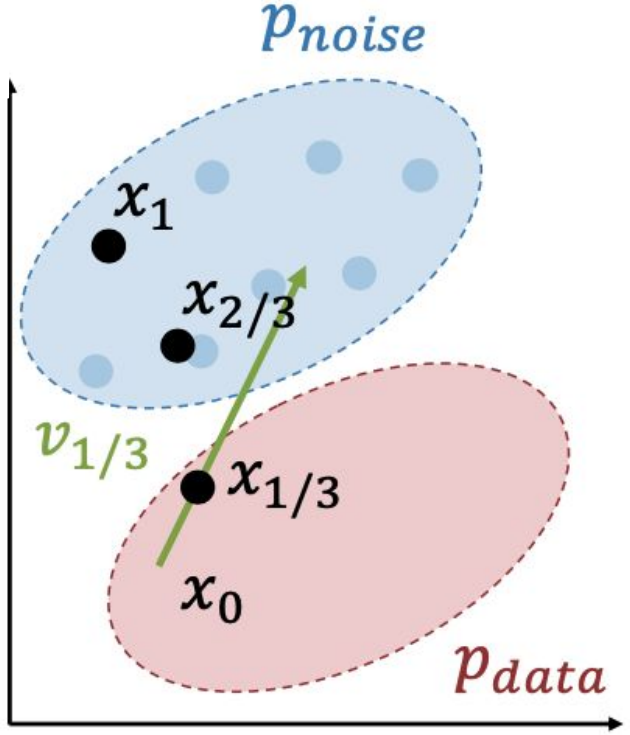
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Diffusion Models: Sampling

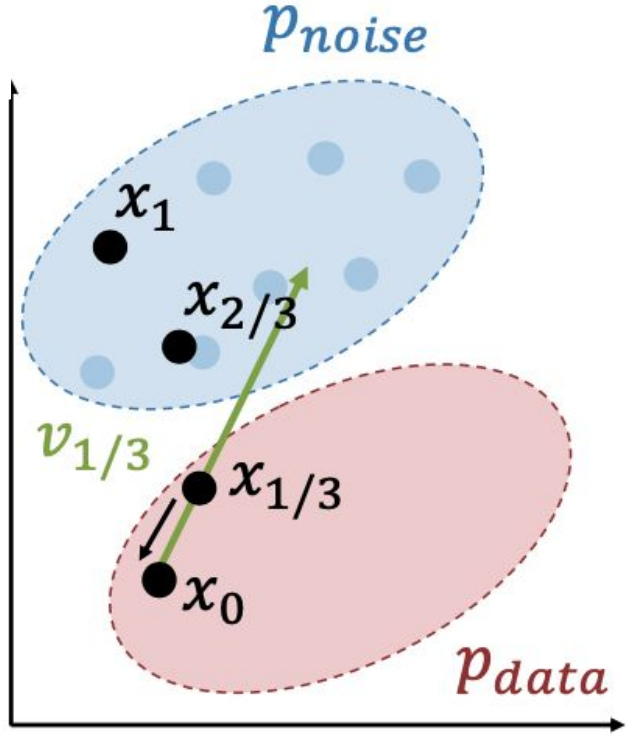
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

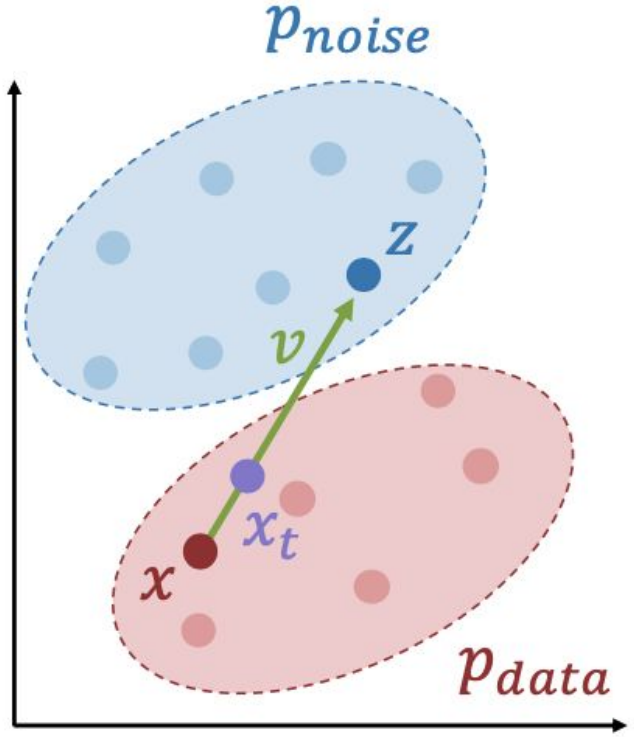
Rectified Flow: Summary

Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

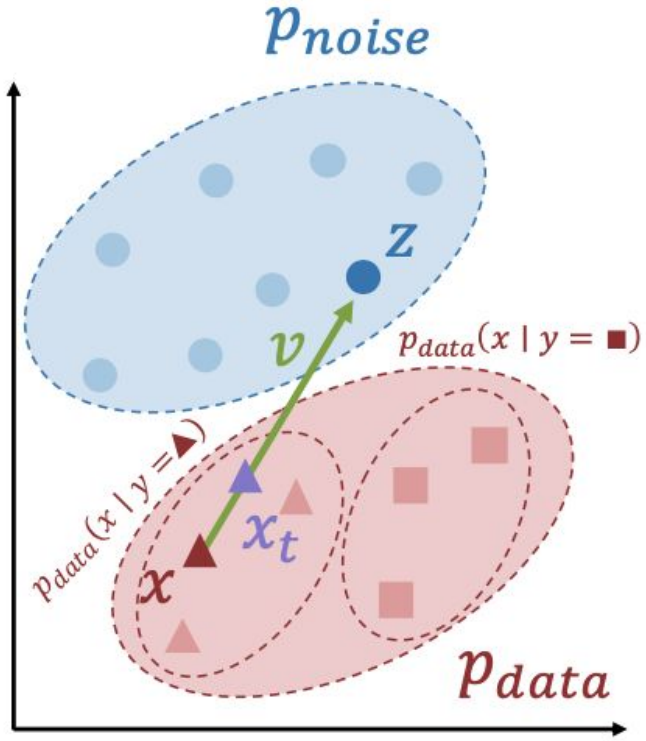
Conditional Rectified Flow

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Slide credit: Justin Johnson

Classifier-Free Guidance (CFG)

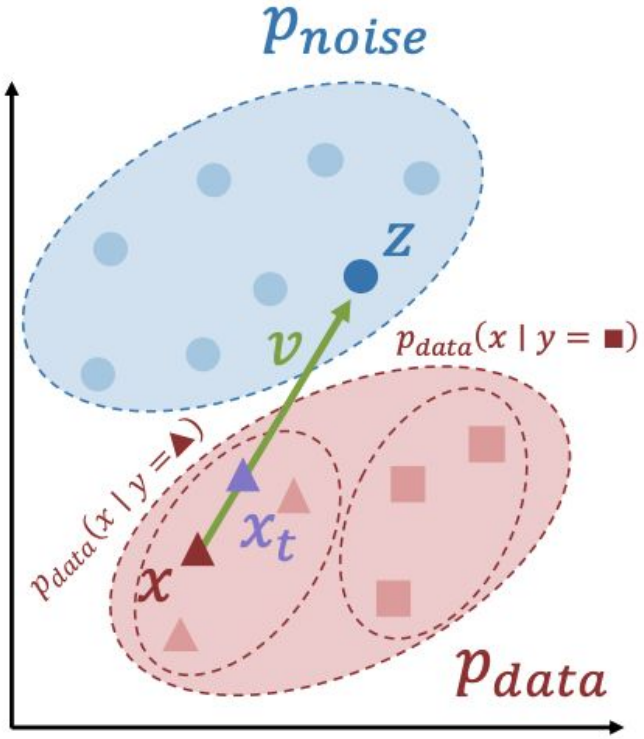
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Randomly drop y during training.

Now the same model is conditional and unconditional!

Can we control how much we "emphasize" the conditioning y?



Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

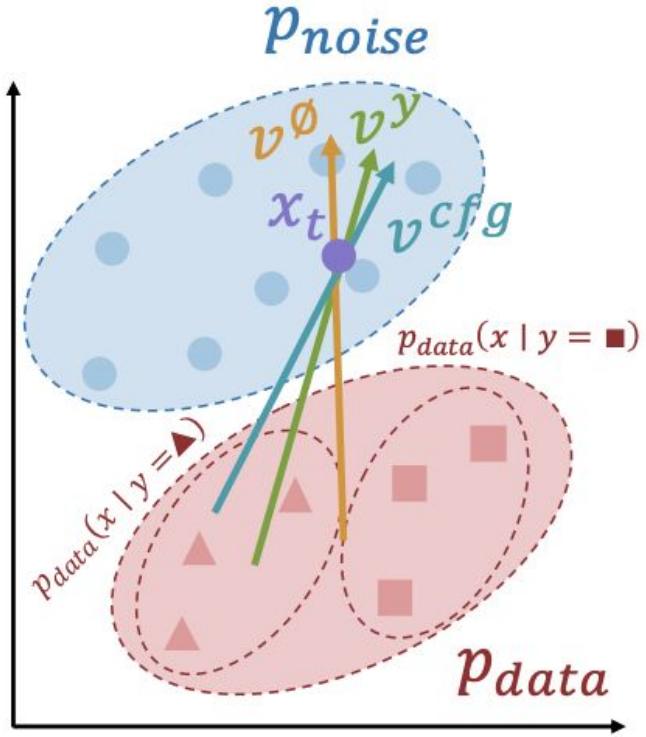
Slide credit: Justin Johnson

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we "emphasize" the conditioning y?



Randomly drop y during training.

Now the same model is conditional and unconditional!
Consider a noisy x_t:

$v^{\emptyset} = f_\theta(x_t, y_\emptyset, t)$ points toward $p(x)$

$v^{y} = f_\theta(x_t, y, t)$ points toward $p(x \mid y)$

$v^{cfg} = (1+w)v^{y} - wv^{\emptyset}$ points more toward $p(x \mid y)$

During sampling, step according to v^{cfg}

Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

Slide credit: Justin Johnson

Classifier-Free Guidance (CFG)

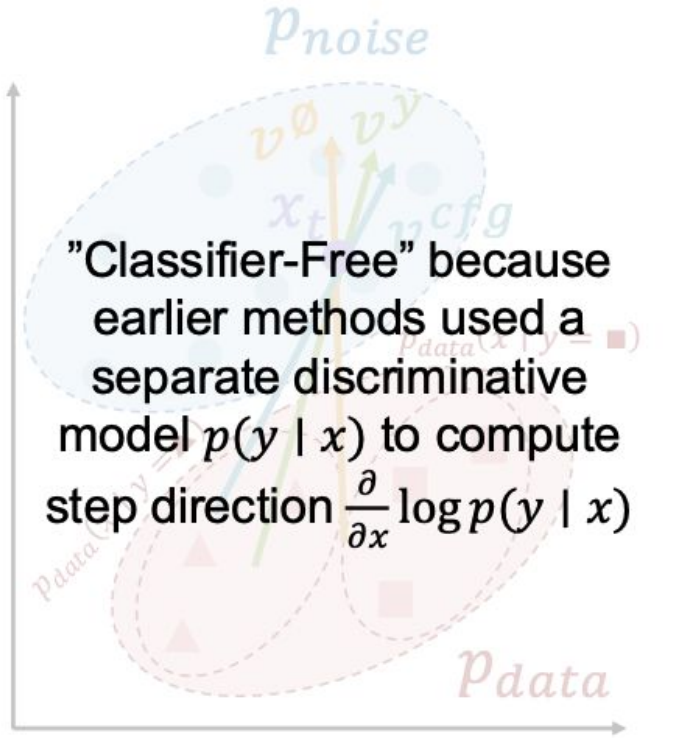
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Can we control how much we "emphasize" the conditioning y?



"Classifier-Free" because earlier methods used a separate discriminative model $p(y \mid x)$ to compute step direction $\frac{\partial}{\partial x} \log p(y \mid x)$

Dhariwal and Nichol, "Diffusion Models beat GANs on Image Synthesis", arXiv 2021
Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

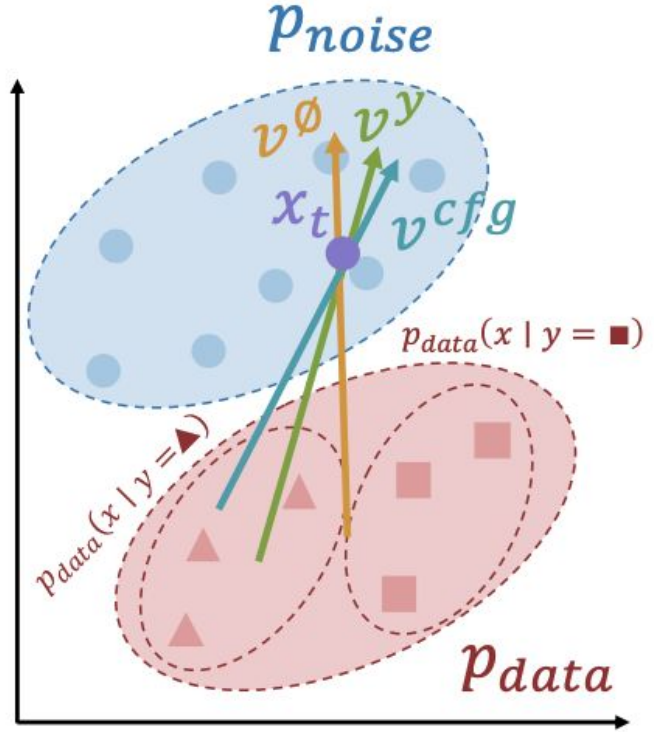
Can we control how much we "emphasize" the conditioning y?

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Used everywhere in practice! Very important for high-quality outputs

Doubles the cost of sampling…



Dhariwal and Nichol, "Diffusion Models beat GANs on Image Synthesis", arXiv 2021
Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

Slide credit: Justin Johnson

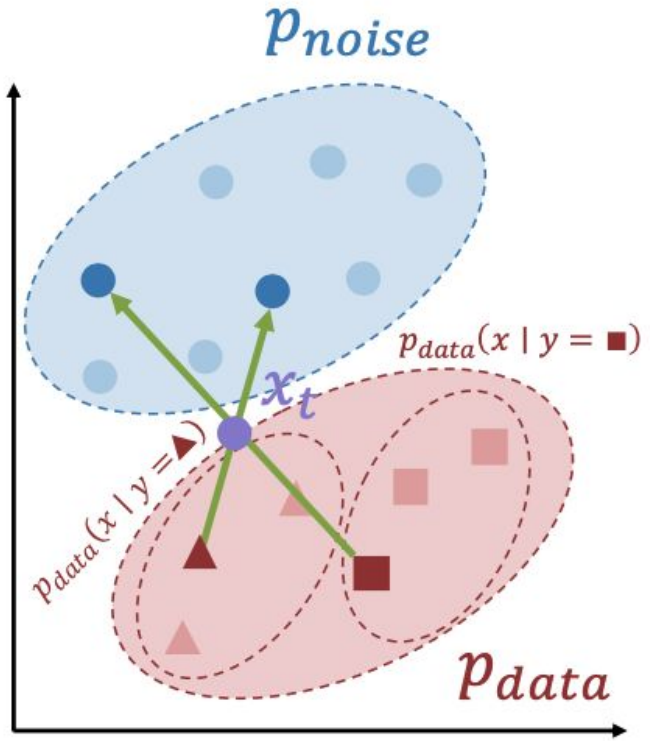
Optimal Prediction

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs (x, z) that give the same x_t; network must average over them

Q: What is the optimal prediction for the network?



Dhariwal and Nichol, "Diffusion Models beat GANs on Image Synthesis", arXiv 2021
Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

Slide credit: Justin Johnson

Optimal Prediction

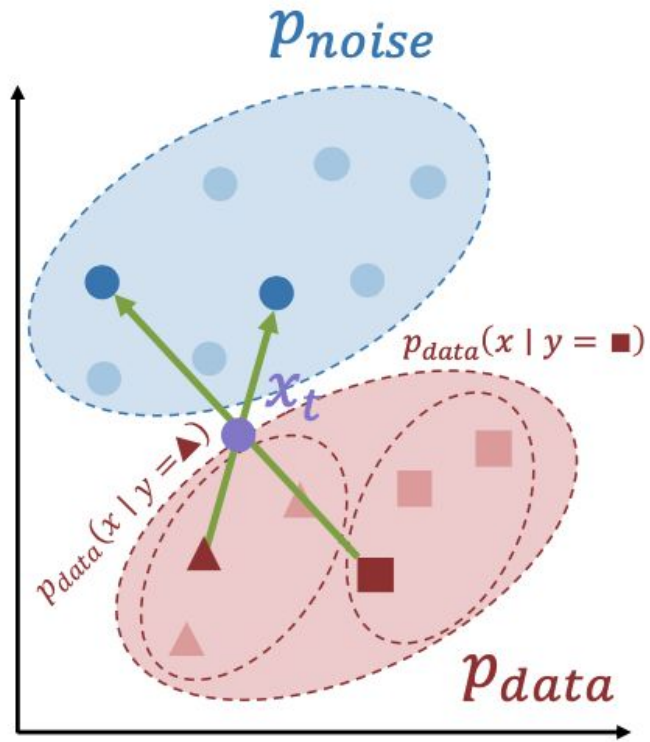
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Q: What is the optimal prediction for the network?

There may be many pairs (x, z) that give the same x_t; network must average over them

Full noise (t=1) is easy: optimal v is mean of p_{data}
No noise (t=0) is easy: optimal v is mean of p_{noise}
Middle noise is hardest, most ambiguous
But we give equal weight to all noise levels!
Solution: Use a non-uniform noise schedule



Dhariwal and Nichol, "Diffusion Models beat GANs on Image Synthesis", arXiv 2021
Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

Slide credit: Justin Johnson

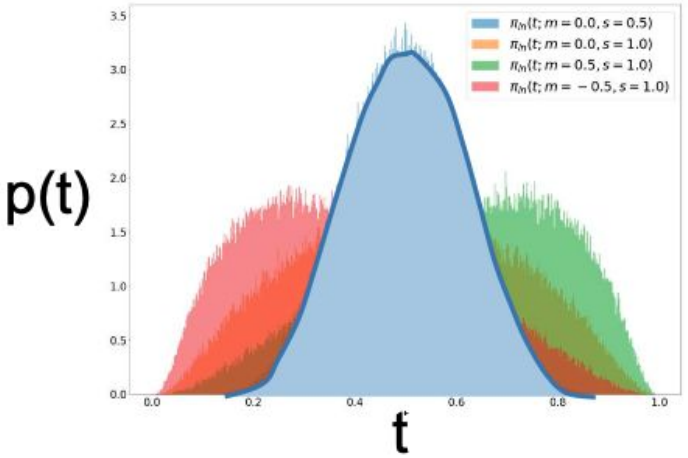
Noise Schedules

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn(()).sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs (x, z) that give the same x_t; network must average over them

Full noise (t=1) is easy: optimal v is mean of p_{data}
No noise (t=0) is easy: optimal v is mean of p_{noise}
Middle noise is hardest, most ambiguous
But we give equal weight to all noise levels!
Solution: Use a non-uniform noise schedule



Put more emphasis on middle noise

Common choice: **logit-normal sampling**

Esser et al, "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", arXiv 2024

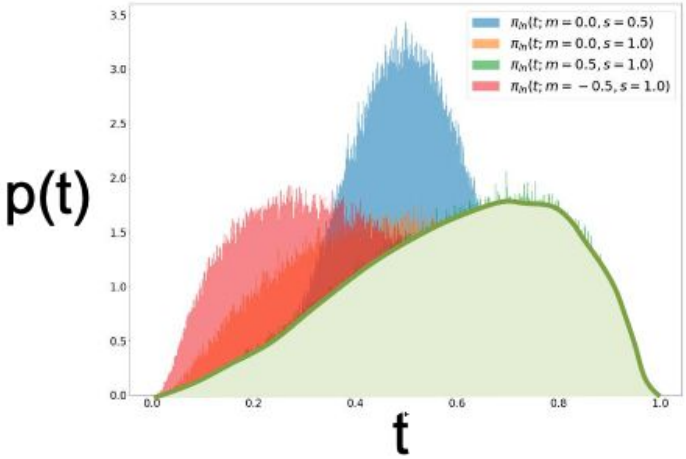
Noise Schedules

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn(()).sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs (x, z) that give the same x_t; network must average over them

Full noise (t=1) is easy: optimal v is mean of p_{data}
No noise (t=0) is easy: optimal v is mean of p_{noise}
Middle noise is hardest, most ambiguous
But we give equal weight to all noise levels!
Solution: Use a non-uniform noise schedule



Put more emphasis on middle noise

Common choice: **logit-normal sampling**

For high-res data, often **shift to higher noise** to account for pixel correlations

Esser et al, "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", arXiv 2024

Slide credit: Justin Johnson

Diffusion: Rectified Flow

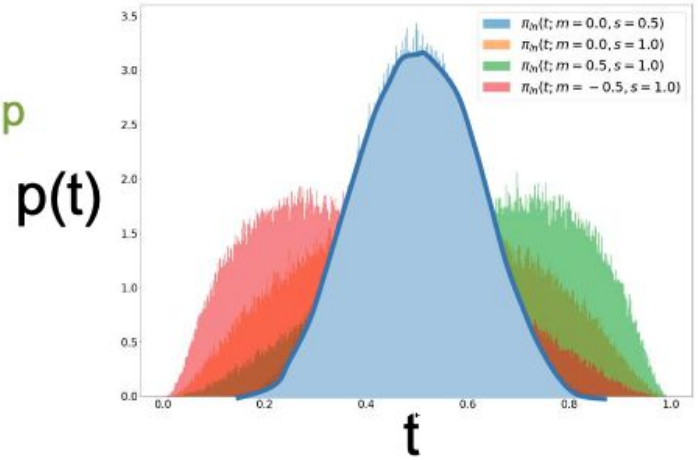
Simple and scalable setup for many generative modeling problems!

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn(()).sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```



Put more emphasis on middle noise

Common choice: **logit-normal sampling**

For high-res data, often **shift to higher noise** to account for pixel correlations

Esser et al, "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", arXiv 2024

Slide credit: Justin Johnson

Diffusion: Rectified Flow

Training

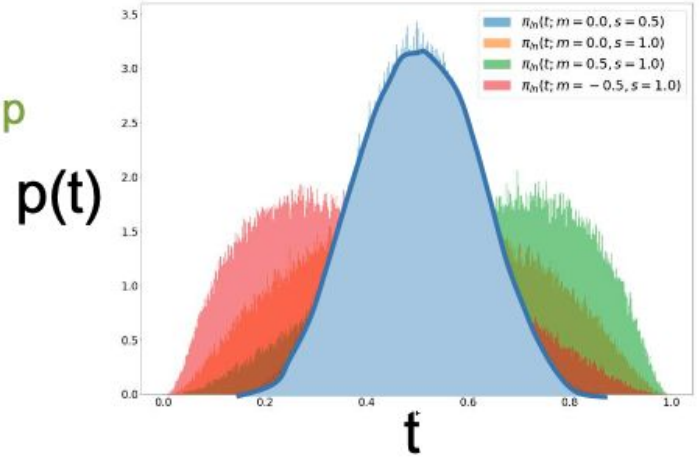
```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn(()).sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Simple and scalable setup for many generative modeling problems!

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Problem: Does not work naively on high-resolution data



Put more emphasis on middle noise

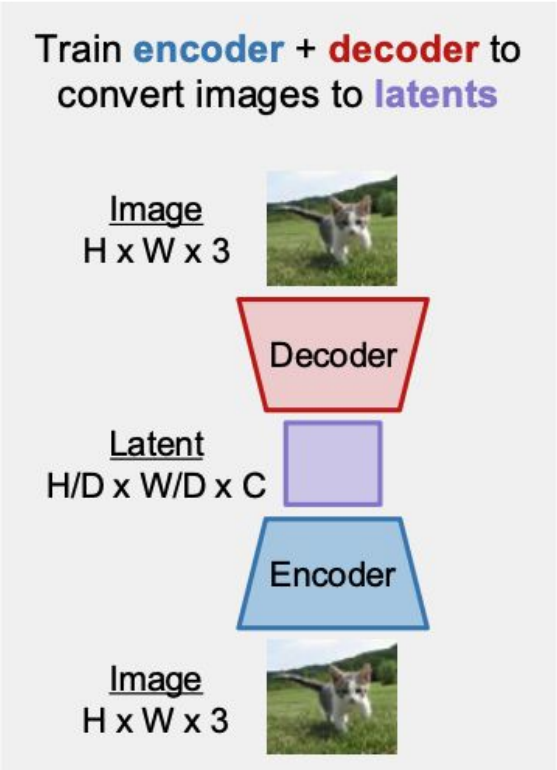
Common choice: **logit-normal sampling**

For high-res data, often **shift to higher noise** to account for pixel correlations

Esser et al, "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", arXiv 2024

Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022



Common setting: D=8, C=16

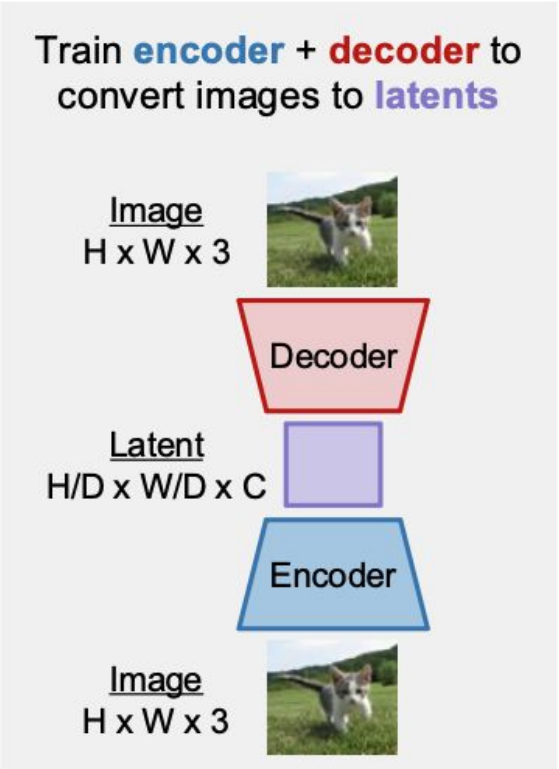
Image: 256 x 256 x 3
=> **Latent**: 32 x 32 x 16

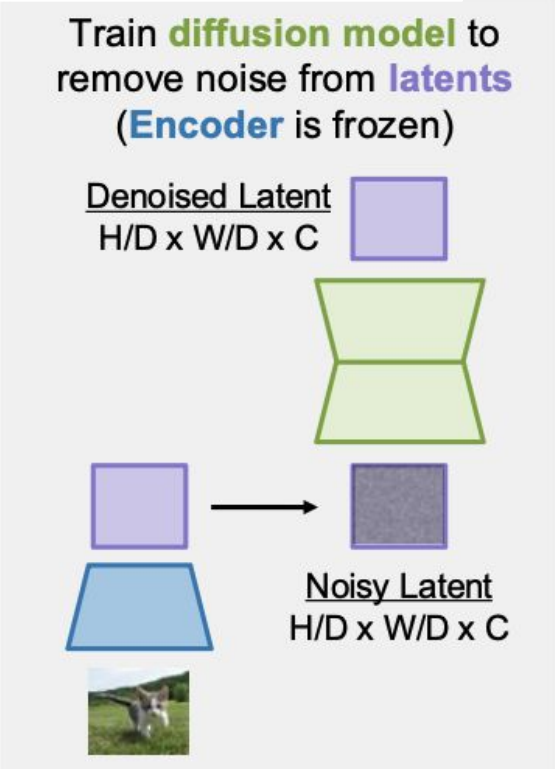
Encoder / Decoder are CNNs with attention

Slide credit: Justin Johnson

Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022



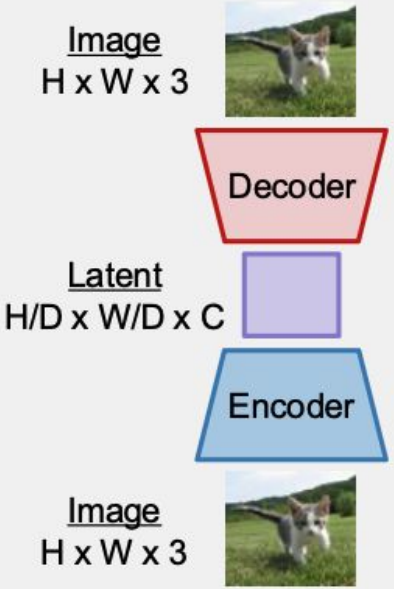


Slide credit: Justin Johnson

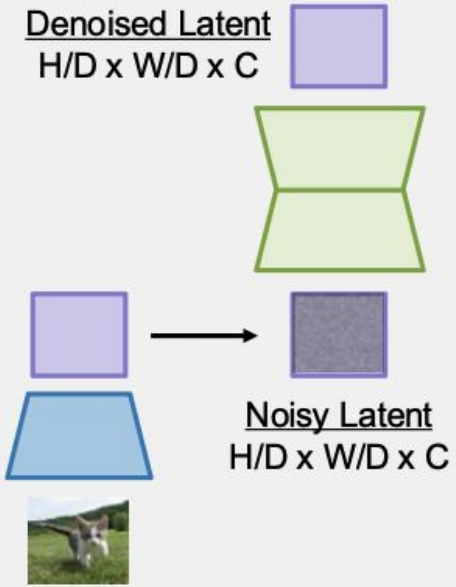
Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Train **encoder** + **decoder** to convert images to **latents**



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)



After training:

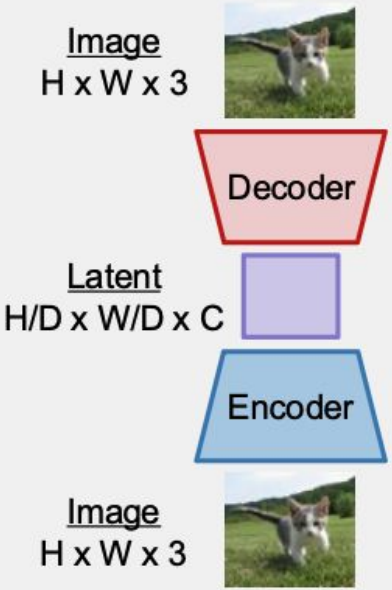
Sample random **latent**

Slide credit: Justin Johnson

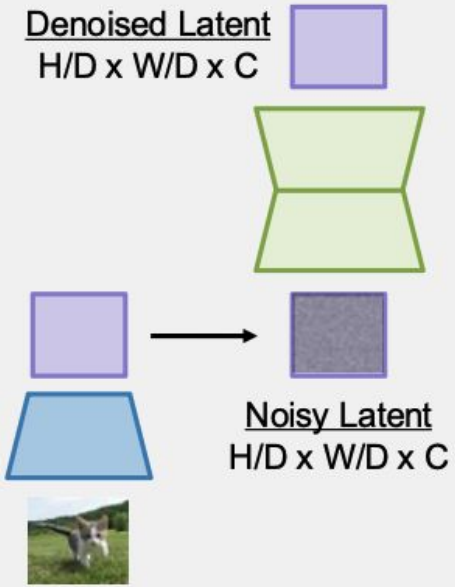
Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Train **encoder** + **decoder** to convert images to **latents**



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)



After training:

Sample random **latent**

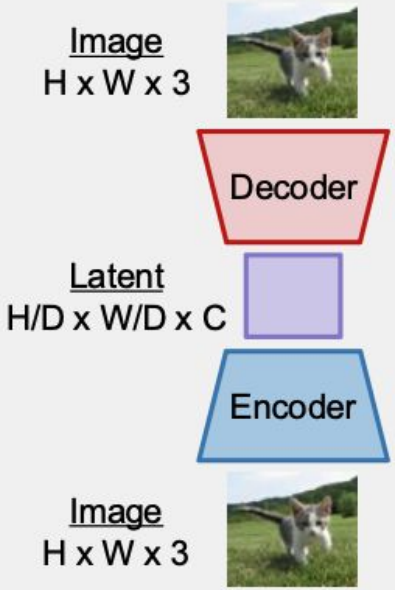
Iteratively apply **diffusion model** to remove noise

Slide credit: Justin Johnson

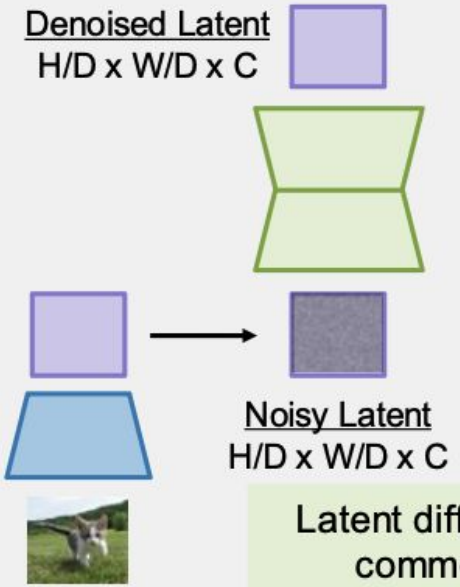
Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Train **encoder** + **decoder** to convert images to **latents**



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)



After training:

Sample random **latent**

Iteratively apply **diffusion model** to remove noise

run **decoder** to get **image**

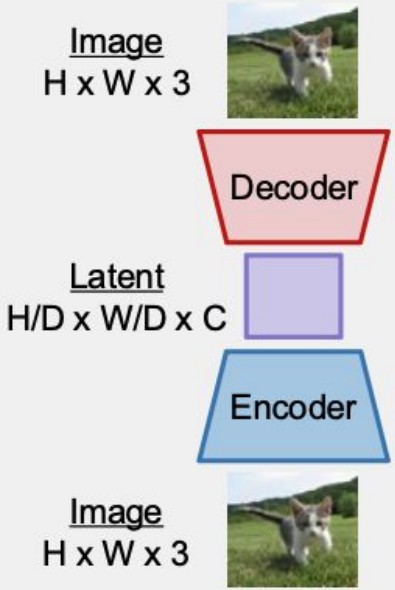
Latent diffusion is the most common form today

Slide credit: Justin Johnson

Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Train **encoder** + **decoder** to convert images to **latents**

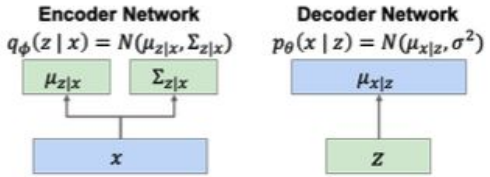


How do we train the encoder+decoder?

Solution: It's a VAE! Typically with very small KL prior weight

Recall: VAE

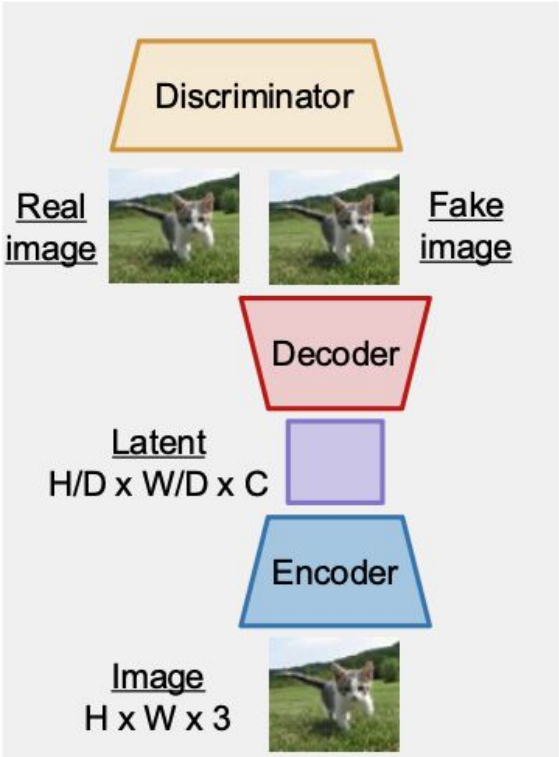
$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$



Slide credit: Justin Johnson

Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022



How do we train the encoder+decoder?

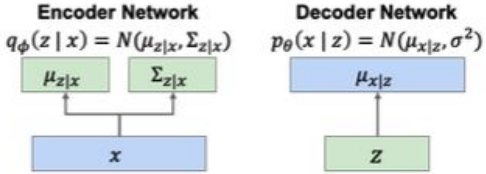
Solution: It's a VAE! Typically with very small KL prior weight

Problem: Decoder outputs often blurry

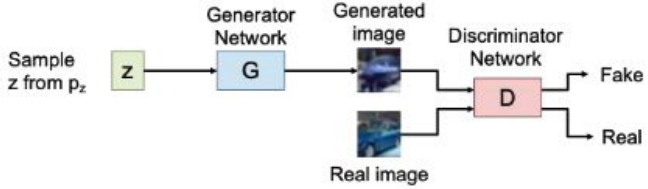
Solution: Add a discriminator!

Recall: VAE

$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$



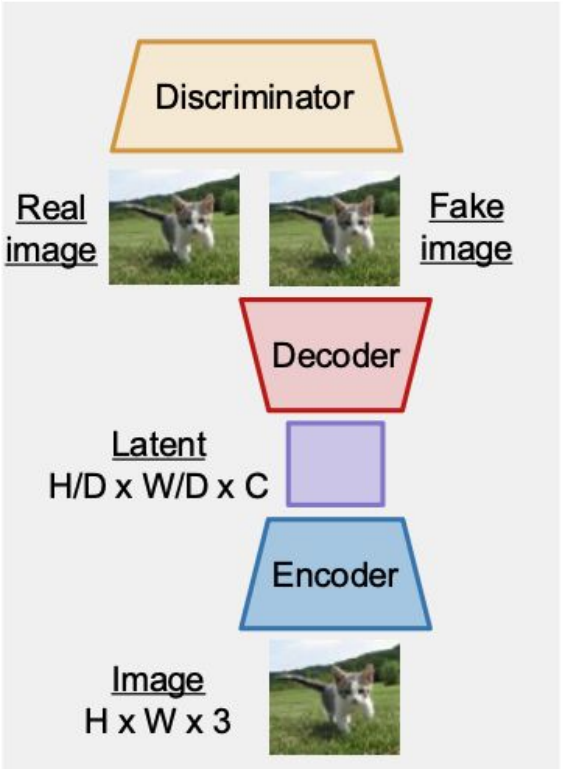
Recall: GAN

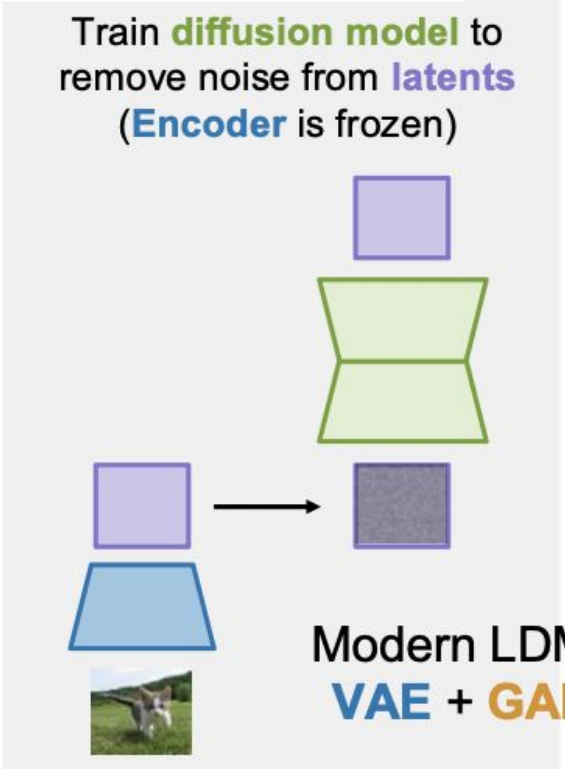


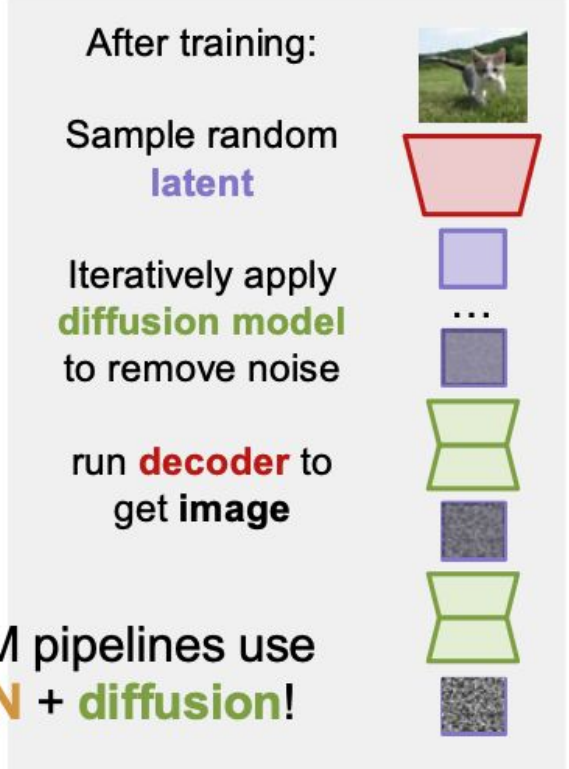
Slide credit: Justin Johnson

Latent Diffusion Models (LDMs)

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

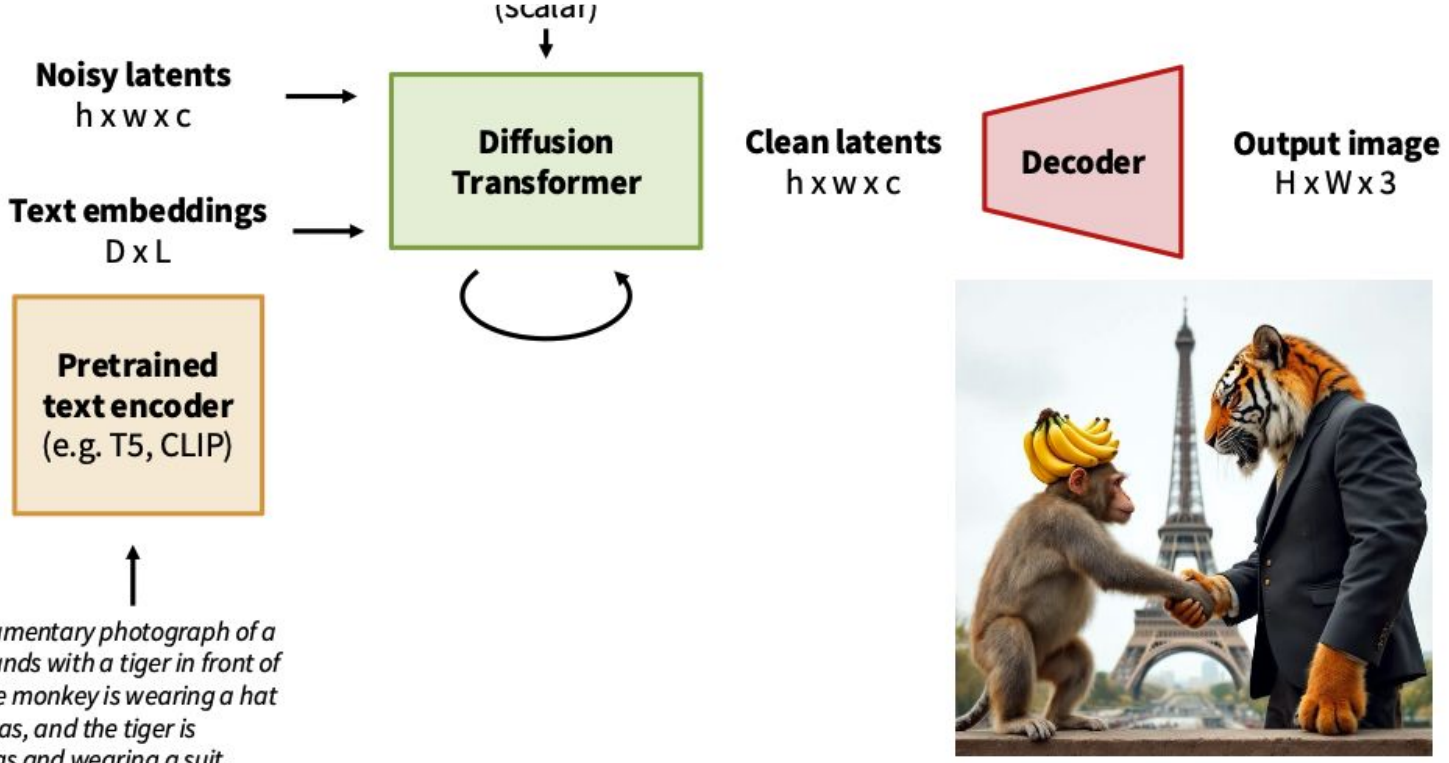






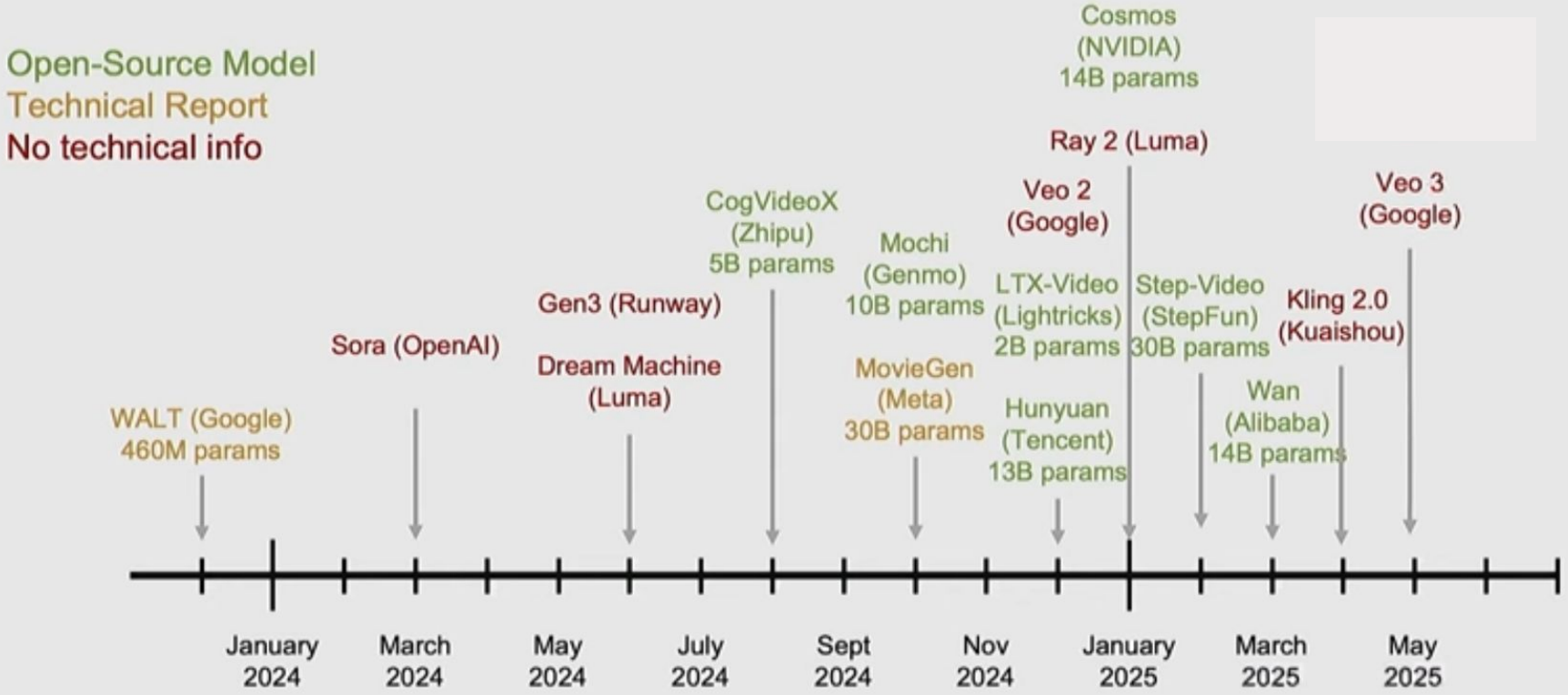
Slide credit: Justin Johnson

Text-to-Image



Slide credit: Justin Johnson

The Era of Video Diffusion Models



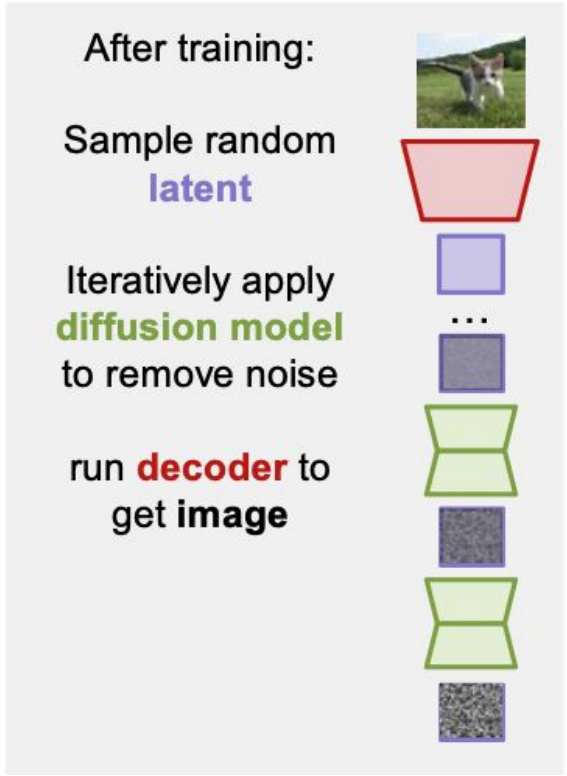
Slide credit: Justin Johnson

Diffusion Distillation

During sampling we need to run the diffusion model many times (~30 – 50 for rectified flow)

This is really slow!

Solution: **distillation** algorithms reduce the number of steps (sometimes all the way to 1), can also bake in CFG



Salimans and Ho, "Progressive Distillation for Fast Sampling of Diffusion Models", ICLR 2022
Song et al, "Consistency Models", ICML 2023
Sauer et al, "Adversarial Diffusion Distillation", ECCV 2024
Sauer et al, "Fast High-Resolution Image Synthesis with Latent Adversarial Diffusion Distillation", arXiv 2024
Lu and Song, "Simplifying, Stabilizing and Scaling Consistency Models", ICLR 2025
Salimans et al, "Multistep Distillation of Diffusion Models via Moment Matching", NeurIPS 2025

Slide credit: Justin Johnson

Generalized Diffusion

Rectified Flow

Sample $x \sim p_{data}, z \sim p_{noise}$
Sample $t \sim p_t$
Set $x_t = (1-t)x + tz$
Set $v_{gt} = z - x$
Compute $v_{pred} = f_\theta(x_t, t)$
Compute loss $\|v_{gt} - v_{pred}\|_2^2$

Generalized Diffusion

Sample $x \sim p_{data}, z \sim p_{noise}$
Sample $t \sim p_t$
Set $x_t = a(t)x + b(t)z$
Set $y_{gt} = c(t)x + d(t)z$
Compute $y_{pred} = f_\theta(x_t, t)$
Compute loss $\|y_{gt} - y_{pred}\|_2^2$

Slide credit: Justin Johnson